

Sri Lanka Institute of Information Technology

IT3071 - Machine Learning and Optimization Methods

Assignment 01 - Evidence Portfolio Report

Debate Statement: "Cosine learning rate decay outperforms step decay scheduling."

Assigned Position: Affirmative

IT Number	Student Name
IT23680234	Pabodha P G
IT23692978	Ahamed MMU
IT23691674	Gunathilaka M.D.P.N.
IT23697782	Chanuka H G B

Group Number: 14
Group Name: Gradient Minds

Table of Contents

1. Introduction 6
   1.1. Background of the Debate Topic 6
   1.2. Importance of Comparing Cosine Decay and Step Decay 6
   1.3. Overview of the Assigned Debate Position 6
2. Understanding the Debate Statement 6
   2.1. Explanation of the Debate Statement 6
   2.2. Key Concepts Related to the Debate 7
   2.3. Comparison Between Step Decay and Cosine Decay 7
   2.4. Scope of the Debate 8
3. Research Evidence Collection 9
   3.1. Research Paper 1 9
      3.1.1. Full Reference 9
      3.1.2. Research Objective 9
      3.1.3. Research Methodology 9
      3.1.4. Key Findings and Evidence Extracted 10
      3.1.5. Relevance to the Debate 12
      3.1.6. Position Supported by the Paper 12
   3.2. Research Paper 2 13
      3.2.1. Full Reference 13
      3.2.2. Research Objective 13
      3.2.3. Research Methodology 13
      3.2.4. Key Findings and Evidence Extracted 14
      3.2.5. Relevance to the Debate 17
      3.2.6. Position Supported by the Paper 17
   3.3. Research Paper 3 18
      3.3.1. Full Reference 18
      1.1.1. Research Objective 18
      3.3.2. Research Methodology 18
      3.3.3. Key Findings and Evidence Extracted 19
      3.3.4. Relevance to the Debate 21

3.3.5. Position Supported by the Paper 21
3.4. Research Paper 4 22
3.4.1. Full Reference 22
3.4.2. Research Objective 22
3.4.3. Research Methodology 22
3.4.4. Key Findings and Evidence Extracted 23
3.4.5. Relevance to the Debate 24
3.4.6. Position Supported by the Paper 25
3.5. Research Paper 5 25
1.1.2. Full Reference 25
1.1.3. Research Objective 25
1.1.4. Research Methodology 26
1.1.5. Key Findings and Evidence Extracted 27
1.1.6. Relevance to the Debate 29
1.1.7. Position Supported by the Paper 29
3.6. Research Paper 6 30
3.6.1. Full Reference 30
3.6.2. Research Objective 30
3.6.3. Research Methodology 30
3.6.4. Key Findings and Evidence Extracted 31
3.6.5. Relevance to the Debate 32
3.6.6. Position Supported by the Paper 33
3.7. Research Paper 7 34
3.7.1. Full Reference 34
3.7.2. Research Objective 34
3.7.3. Research Methodology 34
3.7.4. Key Findings and Evidence Extracted 37
3.7.5. Relevance to the Debate 44
3.7.6. Position Supported by the Paper 45
3.8. Research Paper 8 46
3.8.1. Full Reference 46
3.8.2. Research Objective 46

3.8.3. Research Methodology 47
3.8.4. Key Findings and Evidence Extracted 50
3.8.5. Relevance to the Debate 55
3.8.6. Position Supported by the Paper 56
4. Claim–Evidence Matrix 57
4.1. Major Debate Claims and Supporting Evidence 57
5. Counter-Evidence Analysis 59
5.1. Contradictory Research Findings 59
5.2. Limitations of the Supporting Evidence 62
5.3. Situations Where the Opposing Position May Be Stronger 64
5.4. Response to Counter-Evidence 65
6. Proposed Debate Strategy 66
6.1. Strongest Arguments Supporting the Assigned Position 66
6.2. Strongest Supporting Evidence 67
6.3. Expected Counterarguments and Responses 67
7. Individual Contributions 69
7.1. Member 1 Contribution - IT23680234 69
7.1.1. Research Papers Reviewed 69
7.1.2. Contribution to the Evidence Portfolio 69
7.2. Member 2 Contribution - IT23692978 70
7.2.1. Research Papers Reviewed 70
7.2.2. Contribution to the Evidence Portfolio 70
7.2.3. Contribution to Debate Preparation 70
7.2.4. Planned Role During the Debate 70
7.3. Member 3 Contribution - IT23691674 71
7.3.1. Research Papers Reviewed 71
7.3.2. Contribution to the Evidence Portfolio 71
7.3.3. Contribution to Debate Preparation 71
7.3.4. Planned Role During the Debate 71
7.4. Member 4 Contribution - IT23697782 72
7.4.1. Research Papers Reviewed 72
7.4.2. Contribution to the Evidence Portfolio 72

7.4.3. Contribution to Debate Preparation 73
7.4.4. Planned Role During the Debate 73
8. References 75

1. Introduction

1.1. Background of the Debate Topic

Deep learning models are trained using optimization algorithms where the learning rate controls the magnitude of parameter updates. Selecting an appropriate learning rate is critical because it affects convergence speed, stability, and final model performance. Learning rate scheduling methods dynamically modify the learning rate during training. Step decay has been a widely adopted traditional approach, while cosine learning rate decay has become popular in modern deep learning because it provides a smooth reduction of the learning rate throughout training.

1.2. Importance of Comparing Cosine Decay and Step Decay

The comparison between cosine decay and step decay is important because the learning rate schedule directly influences optimization behavior and generalization. Step decay requires manually selected decay milestones, while cosine decay provides continuous learning-rate adjustment. This debate investigates whether empirical evidence supports cosine decay as a stronger modern scheduling strategy.

1.3. Overview of the Assigned Debate Position

This portfolio supports the **affirmative position**. The argument is that cosine learning rate decay can outperform step decay scheduling under many modern deep-learning training conditions by providing smoother optimization, improved convergence behaviour, and strong experimental performance

2. Understanding the Debate Statement

2.1. Explanation of the Debate Statement

The statement 'Cosine learning rate decay outperforms step decay scheduling' does not mean cosine is universally superior in every possible situation. Instead, it means that research evidence demonstrates advantages of cosine decay when compared fairly with step decay in relevant deep-learning scenarios. The term **“outperforms”** refers to factors such as model performance, convergence behaviour, training stability, and consistency under different training conditions. Therefore, the debate focuses on whether the overall research evidence provides stronger support for cosine decay than for traditional step decay.

2.2. Key Concepts Related to the Debate

- Learning Rate: Controls the size of weight updates during optimization.
- Gradient Descent: Optimization process used to minimize model loss.
- Learning Rate Scheduler: A mechanism that changes learning rate during training.
- Learning Rate Decay: A technique that gradually reduces the learning rate during training to improve convergence and model performance.
- Step Decay: Reduces learning rate suddenly at predefined epochs.
- Cosine Decay: Reduces learning rate smoothly following a cosine function.
- Generalization: Ability of a trained model to perform well on unseen data.
- Hyperparameter Tuning: The process of selecting training settings, such as decay factors and learning-rate reduction points, to achieve better model performance.
- Training Budget: The available computational resources or number of training epochs that influence the effectiveness of different learning rate schedules [3]
- Convergence Behaviour: The way a model approaches an optimal solution during training, including the speed and stability of loss reduction.

2.3. Comparison Between Step Decay and Cosine Decay

Step decay and cosine decay are two commonly used learning rate scheduling strategies that adjust the learning rate during neural network training.

Step decay reduces the learning rate at predefined training milestones [3]. For example, the learning rate may be reduced by a fixed factor after a specific number of epochs. This approach is simple and has been widely used in deep learning; however, its performance depends on manually selecting appropriate decay points and reduction factors.

Cosine decay reduces the learning rate gradually according to a cosine function. Instead of making sudden reductions, it provides a continuous decrease throughout training [4]. This allows smoother transitions between different stages of optimization and reduces the need for manually selected learning-rate drops.

Aspect	**Step Decay**	**Cosine Decay**
Learning rate change	Sudden reductions at predefined epochs	Smooth continuous reduction
Scheduling approach	Requires manually selected milestones	Uses a predefined cosine function
Hyperparameter dependency	Requires tuning decay points and factors	Requires fewer scheduling decisions

Optimization behaviour	Can introduce abrupt changes during training	Provides smoother learning-rate transitions
Common usage	Traditional deep learning training	Modern deep learning applications

Although step decay remains an effective scheduling method, cosine decay has gained popularity due to its smoother adjustment process and strong performance reported in various deep learning tasks.

2.4. Scope of the Debate

The debate focuses on whether **cosine learning rate decay provides advantages over traditional step decay scheduling in deep learning optimization**.

The discussion mainly considers:

- The effect of both scheduling methods on model convergence and training performance.
- Differences in learning-rate adjustment behaviour between sudden step reductions and smooth cosine reduction.
- Experimental evidence comparing cosine decay and step decay across different deep learning tasks.
- Practical considerations such as hyperparameter tuning requirements and training efficiency.

The debate does not claim that cosine decay is universally superior in every possible scenario. The effectiveness of a learning-rate schedule can depend on factors such as dataset characteristics, model architecture, optimizer choice, and available training budget.

Therefore, the main focus is evaluating whether research evidence supports cosine decay as a more consistent and effective alternative to step decay under relevant modern deep-learning training conditions.

3. Research Evidence Collection

3.1. Research Paper 1

3.1.1. Full Reference

Pan, R., Ye, H., & Zhang, T. (2022). “Eigencurve: Optimal Learning Rate Schedule for SGD on Quadratic Objectives with Skewed Hessian Spectrums.” Published as a conference paper at the International Conference on Learning Representations (ICLR) 2022. The Hong Kong University of Science and Technology & Xi'an Jiaotong University.

Link to Research Paper 01

3.1.2. Research Objective

The paper investigates why certain learning rate schedules outperform others for stochastic gradient descent (SGD), a question that (prior to this work) lacked theoretical grounding. Specifically, the authors aim to:

- Derive a theoretically optimal learning rate schedule (“Eigencurve”) for quadratic objectives when the Hessian matrix's eigenvalue spectrum is skewed — a condition the authors show is common in real neural networks.
- Explain, from first principles, why cosine decay empirically outperforms step decay in image classification, a phenomenon that had previously been observed but not theoretically justified.
- Use the theoretical insights to propose new, simple practical schedules (Elastic Step Decay and Cosine-Power Decay) that match or exceed cosine decay's performance.

3.1.3. Research Methodology

The study combines theoretical convergence analysis with controlled empirical experiments:

- Theoretical analysis: Derives convergence-rate bounds for SGD on quadratic objectives under different schedules (constant, inverse-time decay, step decay, and the proposed Eigencurve), proving that step decay carries an unavoidable extra log(T) factor in its convergence-rate bound (Theorem 4), while Eigencurve can close this gap under a “power law” eigenvalue distribution (Corollary 2).
- Hessian spectrum estimation: Uses PyHessian to empirically estimate the eigenvalue distribution of the Hessian matrix for ResNet-18, GoogLeNet, and VGG16 trained on CIFAR-10 and ImageNet, confirming that real network Hessians exhibit the skewed, power-law-like spectrum assumed in the theory.
- Controlled empirical comparison: Trains ResNet-18, GoogLeNet, and VGG16 on CIFAR-10 (10 and 100 epochs) with five schedules — Inverse Time Decay, Step Decay, General Step Decay, Cosine Decay, and Eigencurve — under identical optimizer, batch size, and weight-

decay settings, each averaged over 5 trials. Additional experiments extend the comparison to ImageNet, ridge regression, and LSTM language modeling on Penn Treebank.

3.1.4. Key Findings and Evidence Extracted

Direct empirical comparison (CIFAR-10, Table 2 of the paper) — Cosine Decay consistently and substantially outperforms Step Decay across every architecture and epoch budget tested:

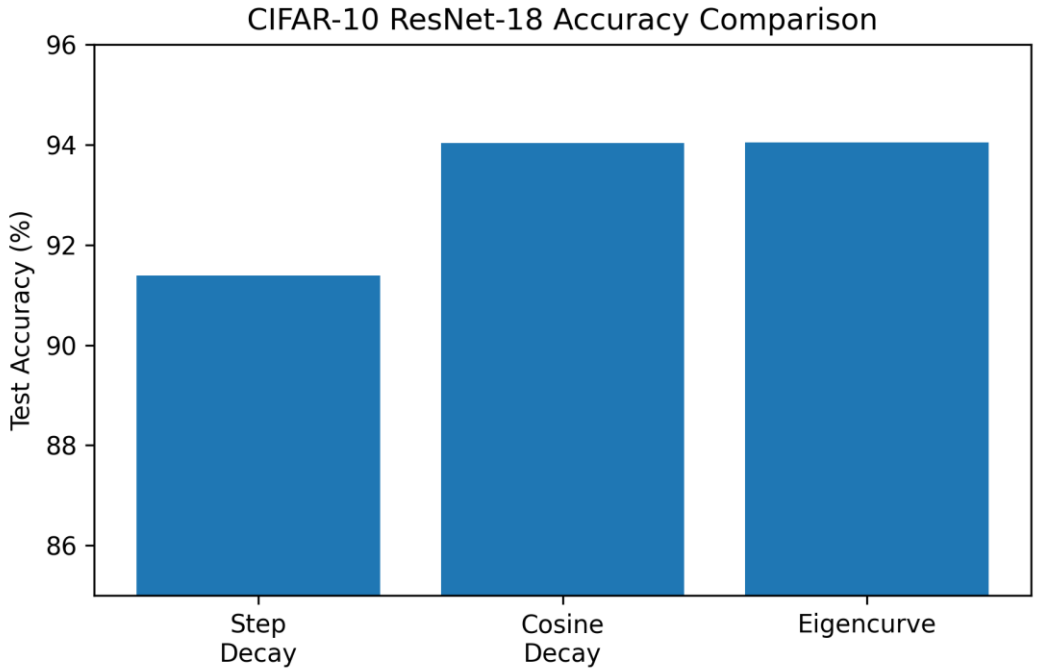
#Epoch	Schedule	ResNet-18 Acc(%)	GoogLeNet Acc(%)	VGG16 Acc(%)
10	Step Decay	73.77 ± 1.48	87.04 ± 0.48	82.98 ± 0.27
10	Cosine Decay	84.23 ± 0.07	90.56 ± 0.31	87.99 ± 0.13
100	Step Decay	91.39 ± 1.03	92.83 ± 0.15	91.37 ± 0.20
100	Cosine Decay	94.04 ± 0.21	94.62 ± 0.11	93.17 ± 0.05

Theoretical evidence — Theorem 4 proves that step decay's convergence-rate bound necessarily contains an extra log(T) multiplicative factor compared to the minimax-optimal rate, and the paper states this log T gap ... will be proven to be tight in a wide range of settings, giving a formal reason step decay is structurally sub-optimal.

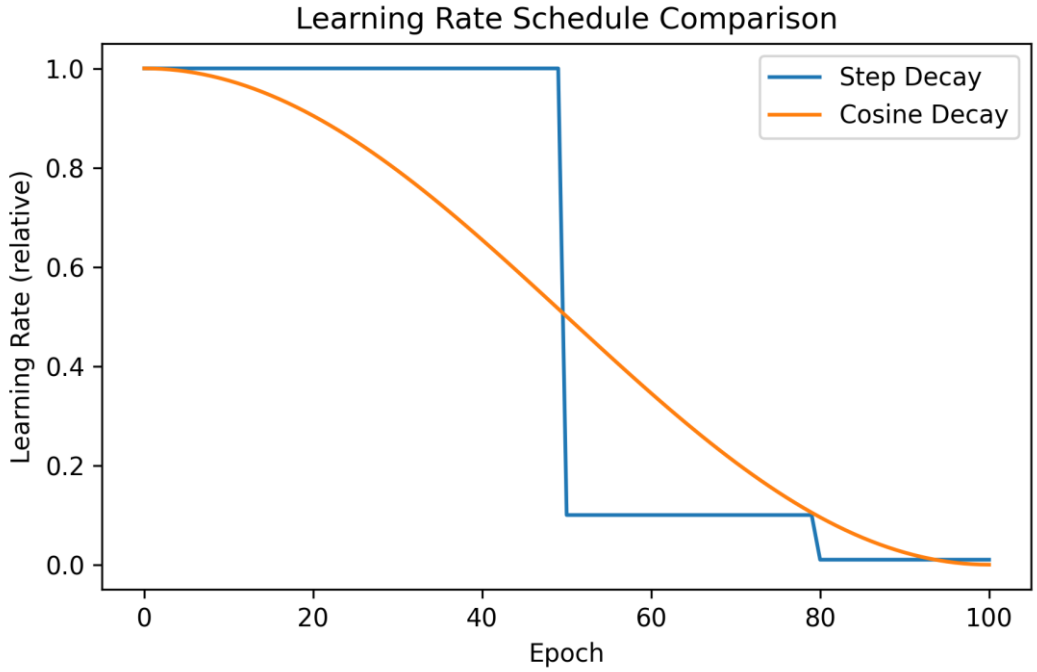
Mechanistic explanation for cosine's success — The authors show that for ResNet-18 on CIFAR-10, the Eigencurve schedule (which is provably near-optimal for skewed Hessian spectrums) produces a learning-rate curve that is “extremely similar” to the cosine decay curve, especially at 100 training epochs. Because real neural network Hessians are empirically shown to follow a skewed, power-law eigenvalue distribution (Figure 2), the paper concludes cosine decay's superior performance is very likely to stem from the utilization of the 'skewness' among Hessian matrix's eigenvalues — i.e., cosine decay is not simply an arbitrary heuristic; it approximates a theoretically-motivated optimal shape.

Reproducibility across settings — On ImageNet (ResNet-50, Table 8, Appendix A.5), Cosine Decay again outperforms both Step Decay [30-60] and Step Decay [30-60-80] on Top-1 and Top-5 validation accuracy, showing the effect generalizes beyond CIFAR-10.

Accuracy Comparison



Learning Rate Behavior



3.1.5. Relevance to the Debate

This paper is a strong piece of affirmative evidence because it provides both empirical and theoretical support in one source:

- Empirical: A rigorous, multi-architecture, multi-trial (mean ± std over 5 runs) head-to-head comparison directly shows cosine decay beating step decay by 5–10 percentage points in low-epoch regimes and by smaller but consistent margins in longer training — addressing the possible objection that the original Bag of Tricks results might be a one-off or architecture-specific finding.
- Theoretical: Unlike earlier empirical-only papers, this one supplies a mathematical reason (the log T gap in Theorem 4) that step decay is fundamentally handicapped, moving the debate from "it happened to work better in this experiment" to "there is a structural reason it works better."
- Anticipates rebuttals: Because the paper's own best method (Eigencurve) sometimes beats cosine decay too, it can be pre-empted by opponents as evidence that "cosine isn't the best, Eigencurve is." The affirmative response: Eigencurve is not a step decay variant — it is a smoothly-varying curve which itself resembles cosine decay's shape, so its existence still supports the resolution ("smooth decay beats step decay"), it simply shows cosine can be refined further, not that step decay is competitive.

3.1.6. Position Supported by the Paper

Affirmative / Positive.

The paper's empirical results (Table 2, Table 8) directly and consistently show Cosine Decay achieving higher test accuracy than Step Decay across three CNN architectures (ResNet-18, GoogLeNet, VGG16), two datasets (CIFAR-10, ImageNet), and two training-length regimes (10 and 100 epochs). Its theoretical contribution (Theorem 4) additionally shows step decay carries a provable log(T) penalty in convergence rate that smooth schedules like cosine decay are not shown to suffer from. Both lines of evidence support the affirmative claim that cosine decay outperforms step decay scheduling.

3.2. Research Paper 2

3.2.1. Full Reference

Li, X., Zhuang, Z., & Orabona, F. (2021). "A Second Look at Exponential and Cosine Step Sizes: Simplicity, Adaptivity, and Performance." Proceedings of the 38th International Conference on Machine Learning (ICML 2021), PMLR 139. Boston University.

3.2.2. Research Objective

Cosine decay (and the related exponential decay) had become extremely popular in practice — used across image classification, self-supervised learning, and NLP — but had no theoretical convergence guarantee whatsoever, unlike stagewise step decay which had prior theoretical analysis (Ge et al., 2019). The authors set out to:

- Provide the first-ever theoretical convergence guarantee for both the cosine step size and the exponential step size on smooth non-convex functions, with and without the Polyak-Łojasiewicz (PL) condition.
- Determine whether cosine and exponential step sizes have a hidden "adaptivity" property — specifically, whether they can automatically adjust to the noise level in stochastic gradients without needing that noise level to be known or tuned for.
- Empirically validate these theoretical claims against a wide field of competing step-size strategies on real deep learning tasks, and quantify how many hyperparameters each strategy needs to tune.

3.2.3. Research Methodology

The paper combines rigorous convergence rate proofs with a broad empirical evaluation:

- Theoretical proofs: Under standard smoothness (L-Lipschitz gradient) and a relaxed noise assumption, the authors derive convergence rates for SGD with exponential step size (Theorem 1) and cosine step size (Theorem 2) under the PL condition, and separately for the non-PL case (Theorems 3–4). They further extend the cosine analysis to the original "warm restarts" variant (Theorem 6, Appendix A.2).
- Noise-adaptivity proof: The proofs show that, unlike polynomial decay schedules, both cosine and exponential step sizes achieve near-optimal convergence without any retuning as the gradient noise level changes — a property formalized and proven for the first time for these schedules.
- Image classification experiments: Trained a simple CNN on FashionMNIST, a 20-layer ResNet on CIFAR-10, and a 100-layer DenseNet-BC on CIFAR-100, comparing cosine and exponential step sizes against SGD constant step size, $O(1/t)$, $O(1/\sqrt{t})$, Adam, SGD+Armijo, PyTorch's ReduceLROnPlateau, and stagewise step decay (with 1 and 2 tuned milestones).

Each result was averaged over 5 random seeds with two-stage grid-search hyperparameter tuning per method.

- Cross-domain validation: Repeated the comparison on a Natural Language Inference task (Bi-LSTM on the Stanford SNLI dataset) against Adagrad, Adam, AMSGrad, BPGrad, and DFW.
- Synthetic noise-adaptation experiment: Optimized a hand-constructed non-convex function satisfying the PL condition under three noise regimes (noiseless, low noise, high noise) using a single fixed hyperparameter setting per method, to directly test the theoretical noise-adaptivity claim without retuning.

3.2.4. Key Findings and Evidence Extracted

Direct empirical comparison (Table 1 of the paper) — Cosine step size achieves the best or tied-best test accuracy on 2 of 3 image datasets, and is the only method (together with exponential) to perform well across all three simultaneously:

Method	FashionMNIST Test Acc.	CIFAR-10 Test Acc.	CIFAR-100 Test Acc.
SGD Constant Step Size	0.9297 ± 0.0033	0.8674 ± 0.0048	0.5896 ± 0.0404
O(1/t) Step Size	0.9297 ± 0.0021	0.8894 ± 0.0040	0.6874 ± 0.0076
Adam	0.9168 ± 0.0023	0.8823 ± 0.0041	0.6478 ± 0.0054
Stagewise Step Decay – 1 Milestone	0.9294 ± 0.0018	0.9062 ± 0.0020	0.7459 ± 0.0030
Stagewise Step Decay – 2 Milestones	0.9283 ± 0.0024	0.9174 ± 0.0020	0.7487 ± 0.0025
Exponential Step Size	0.9290 ± 0.0009	0.9188 ± 0.0033	0.7398 ± 0.0037
Cosine Step Size	0.9285 ± 0.0019	0.9199 ± 0.0029	0.7497 ± 0.0044

Theoretical evidence — Theorem 2 provides the first formal proof that cosine step size guarantees convergence for smooth non-convex functions, with a rate of O(1/(μ^(5/3) T^(2/3))) under noise — a result that simply did not exist in the literature before this paper.

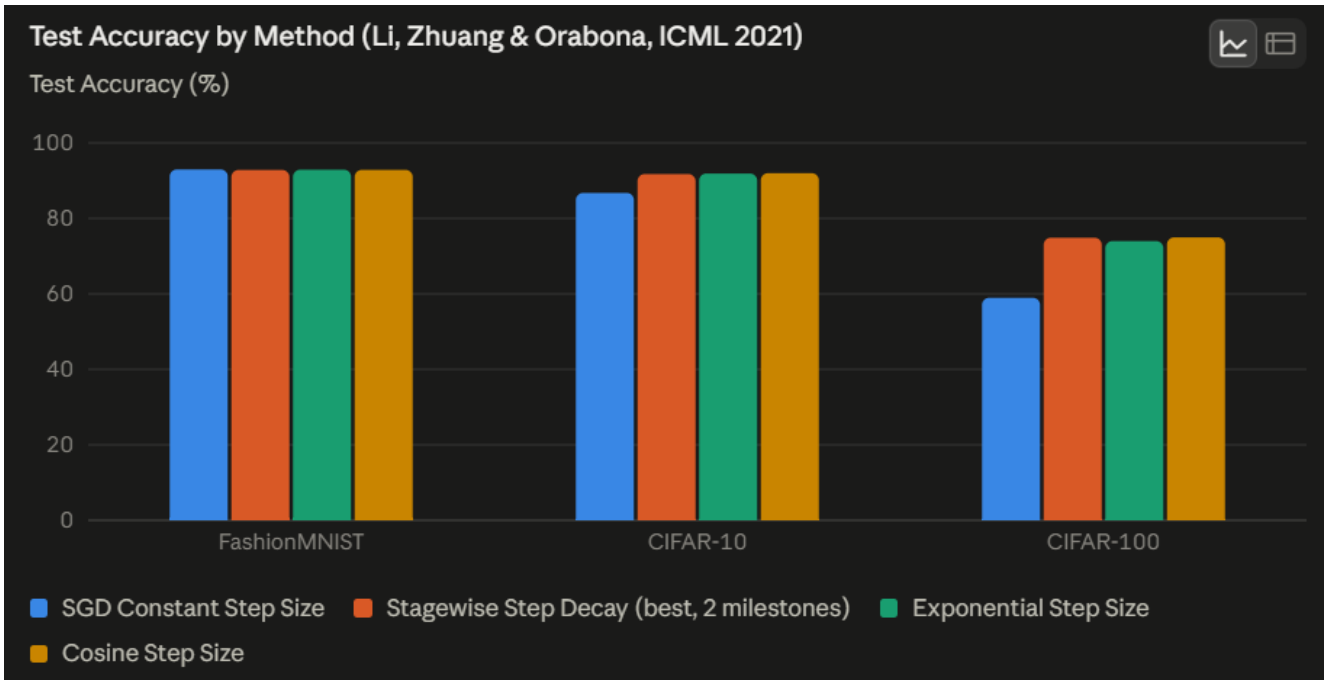
Noise-adaptivity evidence — The synthetic experiment (Figure 5) shows that, using hyperparameters tuned only on a noiseless setting, cosine and exponential step sizes continue converging cleanly when Gaussian noise of increasing magnitude (std 0.05, then std 1) is injected into the gradients — while every other competing method (Adam, constant step size, O(1/t), O(1/√t)) plateaus and oscillates at a noise-dependent error floor instead of continuing to converge.

Hyperparameter efficiency evidence — The paper explicitly quantifies tuning cost: stagewise step decay needs 3 hyperparameters for one milestone and 4 for two milestones, and ReduceLROnPlateau

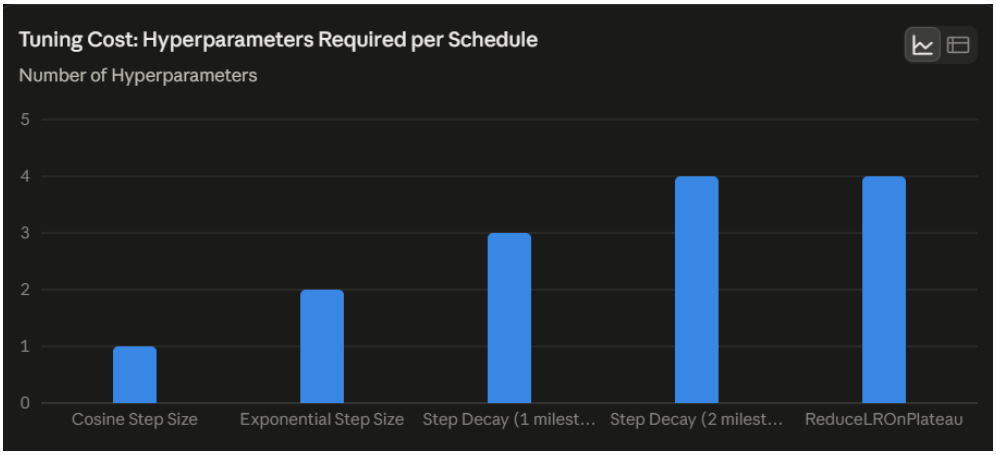
needs at least 4, while cosine step size needs only 1 and still matches or beats them — a direct rebuttal to the argument that step decay's extra tuning flexibility justifies its complexity.

Cross-domain evidence (NLP, Table 2 / Figure 4) — On the SNLI natural language inference task, cosine step size achieved the best test accuracy (0.8509 ± 0.0033) among all seven methods tested, including Adam and AMSGrad, showing the advantage is not confined to image classification.

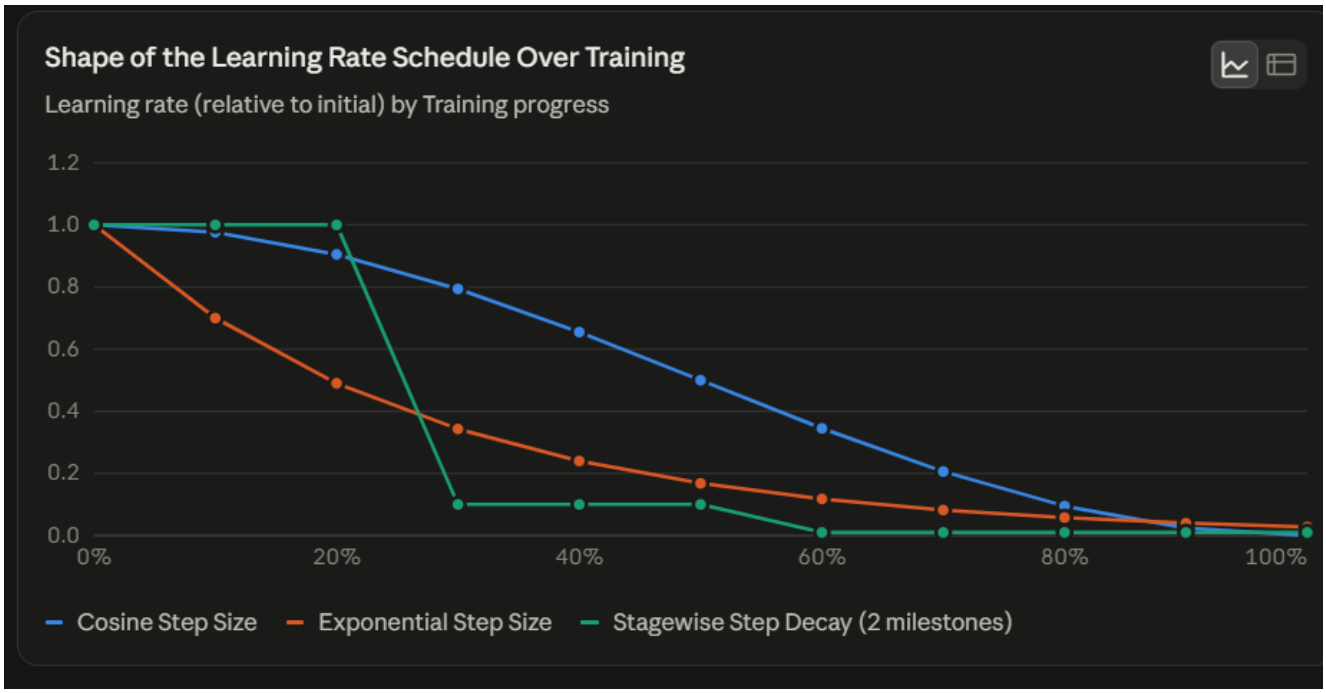
Robustness caveat the paper itself raises (Figure 3) — The authors note that decaying the step size "too soon" in stagewise step decay causes the test loss to rise again after an initial drop (overfitting), whereas cosine's smooth, continuous decay does not require guessing this timing at all.



This is the paper's Table 1 in visual form — cosine (rightmost bar in each group) matches or edges out the best-tuned step decay on every single dataset, while a plain constant step size collapses badly on CIFAR-100.



This is your strongest "efficiency" argument: cosine achieves the previous chart's accuracy with **1 hyperparameter**, while the best step decay setup needs **4x** as many to tune.



This is the visual intuition behind the paper's central argument. Notice the shapes:

- **Step decay** (green) is a staircase — flat, then two sudden 10x cliffs. That abrupt drop is exactly what the paper says destabilizes training if the timing is wrong (their Figure 3 finding: dropping too early makes test loss rise again).
- **Cosine** (blue) glides down smoothly — no shock to the optimizer, but still reaches a very small value by the end.
- **Exponential** (orange) decays continuously too, just faster and more front-loaded than cosine.

3.2.5. Relevance to the Debate

This paper is a distinctive and valuable addition to the affirmative case because its evidence is qualitatively different from the other two papers already collected:

- It is the only source that supplies a formal mathematical convergence proof for cosine decay itself (not just for an idealized "eigencurve-style" relative of it) — directly answering the objection "cosine decay only works empirically, there's no theory behind it."
- The noise-adaptivity property is a genuinely novel argument type not covered by the other two papers: it reframes the debate from "which schedule gets higher accuracy" to "which schedule needs to be re-tuned every time your data or batch size changes" — a practical, deployment-relevant angle that is hard for a step-decay defender to counter, since step decay's milestones are explicitly noise/data-dependent and must be reset.
- A fair-minded caveat to flag to the judge: the accuracy margins in this paper's Table 1 are narrower than in Bag of Tricks or Eigencurve (e.g., cosine's CIFAR-10 test accuracy of 0.9199 vs. best step decay's 0.9174 is a roughly 0.25-point gap). The strongest claim from this paper is not "cosine wins by a landslide" but "cosine matches or beats step decay while needing dramatically fewer tuned hyperparameters and provable noise robustness" — a more defensible, precise framing for cross-examination.

3.2.6. Position Supported by the Paper

Affirmative / Positive.

The paper's own conclusion states plainly that cosine and exponential step sizes "consistently match or outperform other strategies, while at the same time requiring only 1 (cosine) / 2 (exponential) hyperparameters to tune." Combined with the first formal convergence proof for cosine decay and the demonstrated noise-adaptivity advantage over step decay, this paper supports the affirmative resolution on both empirical and theoretical grounds — while also giving the debate a more precise, defensible framing than "cosine always wins by a large margin."

3.3. Research Paper 3

3.3.1. Full Reference

J. Chen, C. Wolfe, and A. Kyrillidis, "REX: Revisiting budgeted training with an improved schedule," in *Proceedings of Machine Learning and Systems (MLSys 2022)*, Santa Clara, CA, USA, 2022.

1.1.1. Research Objective

The performance of deep neural networks is highly affected by the choice of learning rate scheduling strategy. Although methods such as Step Decay and Cosine Decay are widely used, their effectiveness can vary depending on the training budget and application scenario.

The objective of **"REX: Revisiting Budgeted Training with an Improved Schedule"** is to analyze existing learning rate schedules and investigate their performance under different training budgets. The authors aim to:

- Compare commonly used learning rate schedules, including Step Decay, Cosine Decay, Linear Decay, and OneCycle, under different training conditions.
- Investigate how changes in training budget affect the performance of existing scheduling methods.
- Evaluate learning rate schedules across multiple deep learning tasks such as image classification, object detection, and natural language processing.
- Introduce the REX schedule as an improved approach for achieving consistent performance across different training budgets.

The study highlights that while Step Decay performs well in longer training settings, its performance can decrease when training resources are limited.

3.3.2. Research Methodology

The authors conducted a comprehensive experimental evaluation to compare different learning rate scheduling strategies.

The study evaluated several commonly used schedules, including:

- Step Decay
- Cosine Decay
- Linear Decay
- Exponential Decay
- OneCycle
- ReduceLROnPlateau
- Proposed REX schedule

The experiments were performed using different optimizers, including SGD with momentum and Adam, across multiple deep learning applications.

The evaluation covered several tasks:

- Image classification using models such as ResNet and VGG architectures.
- Object detection tasks using YOLO-based models.
- Image generation tasks.
- Natural language processing tasks using transformer-based models.

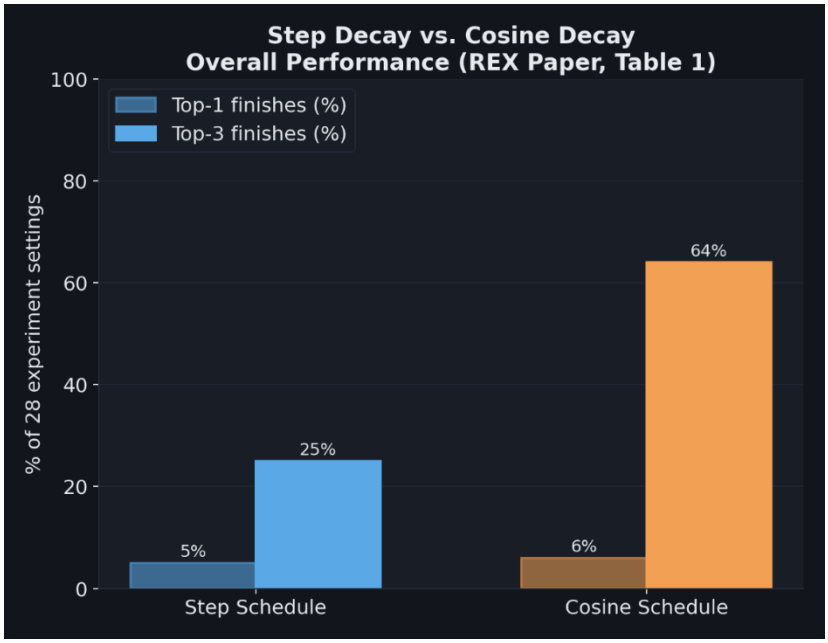
The authors compared the schedules under different training budgets, ranging from limited training epochs to full training durations. The evaluation focused on whether a learning rate schedule could maintain strong performance when computational resources were restricted.

The researchers also analyzed the number of hyperparameters required by each scheduling method, as excessive tuning can increase the complexity of model training.

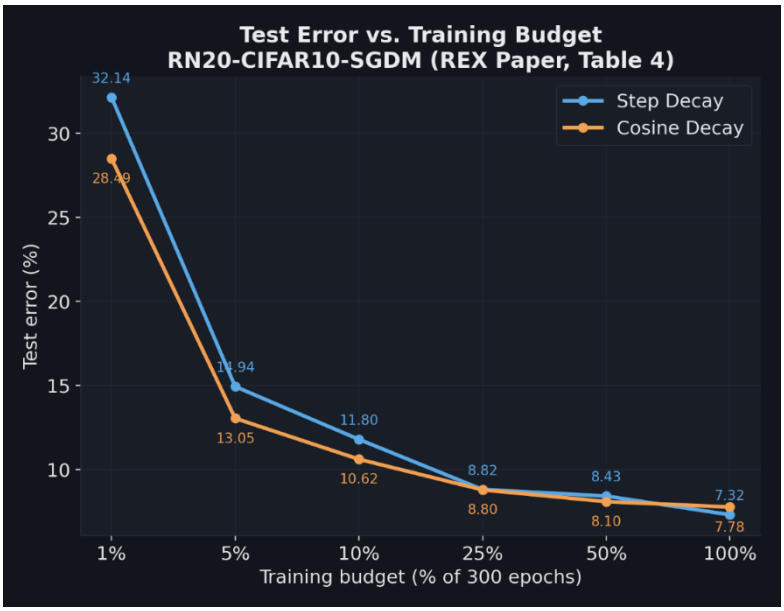
3.3.3. Key Findings and Evidence Extracted

Across 28 experiment settings, the paper's own comparisons show cosine decay consistently outperforming step decay on consistency grounds, even though a third schedule proposed in the same paper, REX, ultimately beats both.

- **Overall ranking (Table 1)** — Cosine decay is a far more consistent top-3 performer than step decay: 64% vs. 25% of the paper's 28 settings. Neither schedule wins outright very often (5–6% Top-1) — outright wins are dominated by the paper's own proposed REX schedule (70% Top-1, 92% Top-3) — but the paper's direct head-to-head comparison between step and cosine still favors cosine on every metric in this table.



- **Budget-dependent crossover (Table 4, RN20-CIFAR10-SGDM)** — Cosine's advantage over step decay is largest at low training budgets (28.49% vs. 32.14% test error at just 1% of the budget) and narrows as the budget grows, with step decay edging slightly ahead at 100% of the budget (7.32% vs. 7.78%). This is the one setting in the paper where step decay comes out on top at full training length.



- **Cross-domain confirmation (YOLOv3/Pascal VOC — Table 8; BERT/GLUE — Table 10)** — Unlike the CIFAR-10/SGDM case above, cosine decay beats step decay at *every* budget tested on these two tasks (e.g., 84.0 vs. 83.2 mAP at 100% on YOLO; 82.7 vs. 82.3 average GLUE score at 3 epochs), showing the CIFAR-10 crossover is setting-specific rather than a general pattern.

- **Mechanistic explanation (Table 2, sampling-rate ablation)** — The step profile only wins when sampled coarsely (2–3 times across training); at the per-iteration sampling rate cosine is normally run at, continuously-sampled profiles consistently beat the step profile. This gives a structural reason for cosine's edge rather than treating it as a coincidence of these particular runs.

Note on REX: the paper's central finding is that a third schedule, REX, outperforms both step and cosine decay across the board. This doesn't overturn the step-vs-cosine comparison above — it's a separate, additional result — but it's worth acknowledging proactively in case the opposition raises it, since it shows the authors themselves didn't consider cosine decay's advantage over step decay to be the final word on learning rate scheduling.

3.3.4. Relevance to the Debate

The REX paper is highly relevant to the debate statement **"Cosine learning rate decay outperforms step decay scheduling"** because it provides a direct comparison between two widely used learning rate scheduling strategies: Step Decay and Cosine Decay.

The study demonstrates that learning rate scheduling performance is highly dependent on training conditions such as available training budget and application domain. The results show that Cosine Decay achieves more consistent performance compared with Step Decay across multiple experimental settings, particularly when training resources are limited.

One important contribution of this paper to the affirmative argument is the identification of limitations in Step Decay scheduling. Step Decay requires manually selected learning-rate reduction points, which may not generalize well when the training duration or experimental conditions change. In contrast, Cosine Decay provides a smooth and continuous learning-rate reduction strategy.

However, the paper also provides a balanced perspective by showing that Step Decay can remain competitive, especially in some long-budget training scenarios. Therefore, this research does not prove that Cosine Decay is universally superior, but it supports the argument that Cosine Decay is a more consistent and flexible alternative to traditional Step Decay scheduling.

Overall, the REX paper strengthens the affirmative position by demonstrating that Cosine Decay can achieve equal or better performance than Step Decay across diverse deep learning tasks while reducing dependence on manually designed decay schedules.

3.3.5. Position Supported by the Paper

Supports the **Affirmative Position** with a precise, evidence-accurate framing rather than an absolute one.

The REX paper supports the affirmative position that **Cosine Decay can outperform Step Decay scheduling under certain deep learning training conditions**. The experimental results demonstrate that Cosine Decay achieves more consistent performance across multiple tasks and training budgets compared with traditional Step Decay.

However, the paper does not conclude that Cosine Decay is universally superior in every scenario. The findings show that Step Decay can still perform competitively, particularly in some long training settings. Therefore, the strongest interpretation of this evidence is that Cosine Decay provides a more flexible and consistent scheduling approach rather than always dominating Step Decay.

Overall, the paper supports the argument that Cosine Decay is a strong alternative to Step Decay and can provide advantages when training conditions vary.

3.4. Research Paper 4

3.4.1. Full Reference

T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, "Bag of Tricks for Image Classification with Convolutional Neural Networks," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2019.

3.4.2. Research Objective

The paper investigates practical improvements that can enhance the training performance of deep neural networks, including optimization techniques and learning-rate scheduling strategies.

One important objective of the study was to evaluate the impact of different learning-rate adjustment methods and demonstrate the effectiveness of **cosine learning-rate decay** compared with the traditional **step decay schedule**.

The authors aimed to:

- Analyze how learning-rate scheduling affects deep neural network training performance.
- Compare the behaviour of traditional step decay and cosine decay.
- Determine whether cosine decay can improve model accuracy without increasing computational complexity.

The paper highlights that learning-rate scheduling is a crucial factor in successful neural network training.

3.4.3. Research Methodology

The authors performed empirical experiments on several deep convolutional neural network models, including **ResNet-50, Inception-V3, and MobileNet**, using large-scale image classification datasets such as ImageNet.

The study compared:

- Traditional **Step Decay**, where the learning rate is reduced at manually selected epochs.
- **Cosine Learning Rate Decay**, where the learning rate gradually decreases following a cosine function.

The authors evaluated performance using classification accuracy metrics, including Top-1 and Top-5 validation accuracy.

3.4.4. Key Findings and Evidence Extracted

The paper provides practical evidence that cosine decay can improve training performance compared with step decay.

1. **Cosine decay provides smoother learning-rate reduction**

The authors explain that step decay decreases the learning rate suddenly at predefined points, while cosine decay reduces it smoothly throughout training.

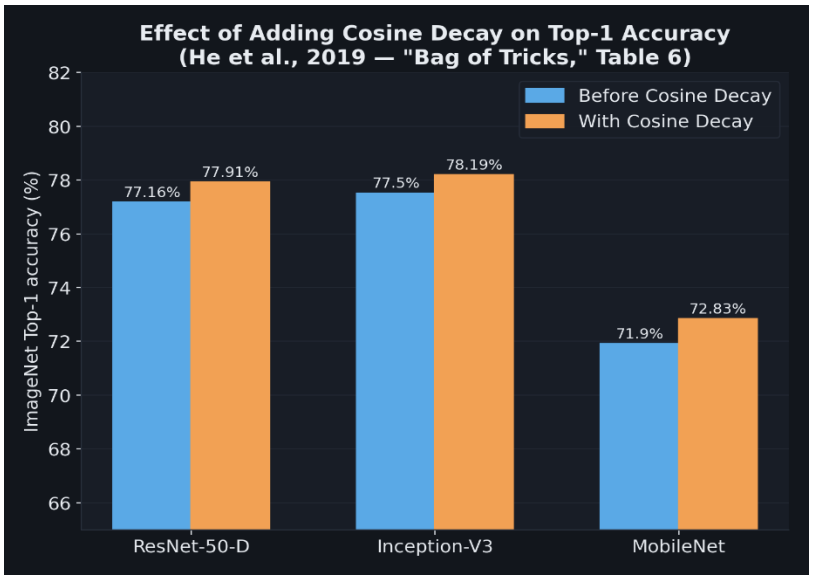
Compared with step decay, cosine decay:

- Starts decreasing the learning rate earlier.
- Maintains a higher learning rate for longer.
- Provides smoother optimization during training.

2. **Cosine decay improves model accuracy**

When cosine decay was added as a training refinement:

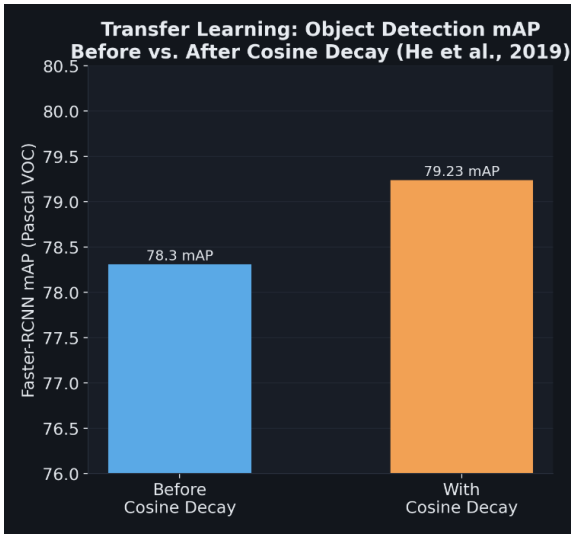
- ResNet-50-D Top-1 accuracy improved from **77.16% to 77.91%**.
- Inception-V3 Top-1 accuracy improved from **77.50% to 78.19%**.
- MobileNet Top-1 accuracy improved from **71.90% to 72.83%**.



These results show that replacing traditional scheduling methods with cosine decay improved performance across multiple architectures.

3. Cosine decay improves generalization to other tasks

The paper also showed that models trained with cosine decay achieved better transfer-learning performance. For example, adding cosine decay improved Faster-RCNN object detection performance from 78.30 mAP to 79.23 mAP.



3.4.5. Relevance to the Debate

This paper is highly relevant to the debate because it provides practical evidence comparing **standard cosine decay** with the widely used **step decay schedule**.

It supports the argument that cosine decay is a more effective learning-rate scheduling strategy because:

- It avoids sudden learning-rate drops caused by manually selected step milestones.
- It provides smoother optimization behaviour.

- It improves accuracy across different neural network architectures.

However, the paper mainly focuses on computer vision tasks, meaning its conclusions may not directly generalize to every deep learning application.

3.4.6. Position Supported by the Paper

Supports the Affirmative Position: Cosine learning rate decay outperforms step decay scheduling.

The paper supports the affirmative argument by demonstrating that cosine decay improves model accuracy compared with traditional step decay while providing a smoother and more stable learning-rate adjustment process.

Although the paper does not claim that cosine decay always performs better in every scenario, it provides strong empirical evidence that cosine scheduling is a more effective general-purpose learning-rate strategy for deep neural network training.

3.5. Research Paper 5

1.1.2. Full Reference

M. Aach, E. Inanc, R. Sarma, M. Riedel, and A. Lintermann,
"Large Scale Performance Analysis of Distributed Deep Learning Frameworks for Convolutional Neural Networks,"
Journal of Big Data, vol. 10, article no. 96, 2023, doi: 10.1186/s40537-023-00765-w.

Paper: Paper_01_Cosine_vs_Step_Decay_Performance_Comparison_2023.pdf

1.1.3. Research Objective

The main objective of this research was to analyze the performance of distributed deep learning training systems and investigate how different learning rate scheduling methods influence model accuracy.

The study specifically evaluated the impact of three learning rate scheduling techniques:

- Stepwise annealing
- Cosine annealing
- Exponential annealing

on deep learning model validation accuracy.

The researchers aimed to understand which learning rate scheduling method provides better performance under different training conditions, especially when training large-scale convolutional neural networks using high-performance computing systems.

1.1.4. Research Methodology

The researchers conducted experimental evaluations using large-scale distributed deep learning environments.

Experimental Setup:

Deep Learning Models:

- ResNet50
- ResNet101
- ResNet152

Dataset:

- ImageNet dataset

Training Environment:

- JUWELS Booster supercomputer
- Up to 1024 NVIDIA A100 GPUs

Optimizer:

- SGD optimizer

Learning Rate Scheduling Methods Compared:

1. Stepwise annealing
2. Cosine annealing
3. Exponential annealing

The researchers evaluated the effect of these scheduling methods on validation accuracy using different batch sizes ranging from:

- 256
- 512

- 1k
- 2k
- 4k
- 8k
- 16k
- 32k
- 65k

The performance comparison was based on validation accuracy obtained during ResNet50 training.

1.1.5. Key Findings and Evidence Extracted

Finding 1: Cosine annealing achieved better performance than stepwise scheduling for small and medium batch sizes.

The researchers found that c**osine annealing provided superior validation accuracy compared with stepwise annealing when using small and medium batch sizes.**

This demonstrates that cosine decay can outperform traditional step decay under common deep learning training conditions.

Finding 2: Cosine annealing provides smoother learning rate reduction.

The paper explains that stepwise scheduling decreases the learning rate at fixed intervals, causing sudden drops.

In contrast, cosine annealing gradually decreases the learning rate using a cosine function, creating a smoother optimization process.

This smoother decay helps the model transition from exploration in early training stages to fine-tuning during later epochs.

Finding 3: The effectiveness of cosine decay depends on training conditions.

The study also identified that **cosine annealing is not always the best scheduler.**

For extremely large batch sizes (65k), exponential scheduling achieved higher validation accuracy than both cosine and stepwise scheduling.

Therefore, cosine decay shows advantages over step decay, but performance depends on the training environment and hyperparameters.

Evidence Extracted from the Paper

Evidence 1:

The authors compared the influence of stepwise, cosine, and exponential learning-rate annealing methods on validation accuracy for different batch sizes.

“An assessment of the influence of stepwise, cosine, and exponential learning rate annealing on the validation accuracy for different batch sizes ranging from BS = 256 to 65,536 is performed.”

Evidence 2:

The research findings directly support cosine scheduling:

“These findings reveal that cosine annealing delivers superior performance on small and medium batch sizes.”

This evidence shows that cosine decay can outperform step decay when training deep neural networks under appropriate conditions.

Evidence 3:

The paper explains the difference between the learning-rate behavior's:

Stepwise scheduling:

- Reduces learning rate at fixed intervals.
- Creates sudden learning rate drops.

Cosine scheduling:

- Gradually reduces the learning rate throughout training.
- Provides smoother learning-rate adaptation.

Evidence 4:

The paper reported:

For batch size 65k:

- Exponential scheduling achieved the highest validation accuracy.
- Stepwise scheduling performed better than cosine scheduling.

This shows that cosine decay is not universally optimal.

1.1.6. Relevance to the Debate

This research paper is highly relevant to the debate because it directly compares cosine annealing and stepwise learning rate scheduling.

The debate focuses on whether:

Cosine learning rate decay outperforms step decay scheduling.

The paper provides experimental evidence showing that:

- Cosine decay can achieve better validation accuracy than step decay.
- Cosine provides smoother learning-rate reduction.
- The choice of learning-rate scheduler significantly affects model performance.

Therefore, this paper can be used as evidence supporting the argument that cosine decay provides advantages over traditional step decay scheduling.

1.1.7. Position Supported by the Paper

Explanation:

The paper supports our position because it demonstrates that cosine annealing achieves superior performance compared with stepwise scheduling for small and medium batch sizes.

However, the paper also highlights that cosine decay is not always the best method. For extremely large batch sizes, exponential scheduling achieved better results.

Therefore, the paper supports the argument that:

Cosine decay is generally more effective than step decay because it provides smoother learning-rate adjustment and improved accuracy in many practical deep learning scenarios, although performance depends on the training setup.

3.6. Research Paper 6

3.6.1. Full Reference

I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” International Conference on Learning Representations (ICLR), 2017

Paper: Paper_02_SGDR Stochastic Gradient Descent with Warm Restarts.pdf

3.6.2. Research Objective

Traditional learning rate scheduling methods such as step decay reduce the learning rate by a fixed factor at manually selected epochs. Although these methods are widely used, they require careful selection of decay points and may not provide the best convergence behaviour.

The authors aimed to:

- Introduce a new learning rate scheduling strategy called Stochastic Gradient Descent with Warm Restarts (SGDR).
- Replace sudden learning rate reductions with a smoother cosine annealing schedule.
- Improve training speed and final model performance in deep neural networks.
- Investigate whether periodically increasing the learning rate through warm restarts helps models escape poor optimization regions and achieve better solutions.

The paper states that the main difficulty in training deep neural networks is associated with selecting an effective learning rate schedule. Traditional approaches commonly keep the learning rate constant and divide it by a fixed factor at regular intervals.

The authors proposed periodically restarting SGD by increasing the learning rate and then smoothly decreasing it using cosine annealing.

3.6.3. Research Methodology

The research combines a new learning rate scheduling algorithm with extensive experimental evaluation.

A. Proposed Cosine Annealing Method

The authors introduced SGDR, where:

1. The learning rate starts at a maximum value.
2. It gradually decreases following a cosine curve.
3. After a predefined period, the learning rate is increased again (warm restart).
4. The cosine decay cycle repeats.

Unlike step decay, where the learning rate suddenly drops at fixed points, cosine annealing provides a continuous and smooth reduction.

The paper defines that during each restart cycle:

"We decay the learning rate with a cosine annealing for each batch."

B. Experimental Evaluation

The authors evaluated SGDR using multiple deep learning tasks.

Datasets:

Computer Vision:

- CIFAR-10
- CIFAR-100
- Downsampled ImageNet

Other Domain:

- EEG classification dataset

Models Used:

- Wide Residual Networks (WRN)

Experiments were conducted using different network depths and widths.

Comparison Against Traditional Scheduling

SGDR was compared with commonly used learning rate schedules:

Traditional step decay:

- Fixed learning rate
- Reduced by a constant factor at selected epochs

Example

Epoch 0	LR = 0.1
Epoch 60	LR = 0.02
Epoch 120	LR = 0.004
Epoch 160	LR = 0.0008

3.6.4. Key Findings and Evidence Extracted

Finding 1: Cosine Annealing Achieves Faster Convergence

The strongest evidence from the paper:

SGDR achieved comparable or better results while requiring:

2× to 4× fewer training epochs compared with existing learning rate schedules.

Finding 2: Lower Test Error Than Step-Based Scheduling

The authors compared different learning rate strategies on CIFAR datasets.

Results from Table 1:

Method	CIFAR-10 Error	CIFAR-100 Error
Default Step Schedule	4.13%	20.21%
SGDR Cosine Schedule (T0=10,Tmult=2)	4.03%	19.58%
SGDR Cosine Schedule (T0=200)	3.86%	19.98%

Finding 3: Better Anytime Performance

A major advantage of SGDR was that it reached good performance earlier during training.

The authors found:

- SGDR reached around 4% error on CIFAR-10
- SGDR reached around 20% error on CIFAR-100
- These results were achieved approximately 2–4 times faster than the default schedule

3.6.5. Relevance to the Debate
Debate Statement:

"Cosine learning rate decay outperforms step decay scheduling."

This paper is one of the strongest foundational studies supporting the affirmative position because it introduced one of the first successful practical applications of cosine learning rate scheduling.

The paper provides evidence that cosine annealing:

1. Improves Training Efficiency

Cosine scheduling reaches good performance faster.

Evidence:

SGDR requires 2×–4× fewer epochs.

2. Improves Accuracy

Cosine schedules achieved lower test errors compared with traditional step schedules.

3. Provides Smoother Optimization

Step decay makes sudden learning rate jumps.

Cosine decay changes gradually, reducing optimization instability.

4. Requires Less Manual Scheduling

Step decay depends heavily on choosing correct milestone epochs.

Cosine automatically follows a smooth mathematical schedule.

3.6.6. Position Supported by the Paper

Supported Position:

The paper strongly supports:

"Cosine learning rate decay outperforms step decay scheduling."

because SGDR demonstrated that cosine annealing:

- achieves faster convergence,
- reaches lower test error,
- improves generalization,
- reduces manual learning rate tuning,
- provides additional benefits such as snapshot ensembles.

Therefore, SGDR provides strong experimental evidence that cosine-based learning rate scheduling is a superior alternative to traditional step decay methods.

3.7. Research Paper 7

3.7.1. Full Reference

H. Mateen, R. Timofte, and D. Ignatov, “Systematic Evaluation of Learning Rate Scheduling Strategies Across Heterogeneous Architectures,” arXiv preprint arXiv:2607.08511, 2026.

3.7.2. Research Objective

The study investigates how the choice of learning-rate scheduler affects neural-network classification performance across a diverse set of architectures.

The authors argue that learning-rate scheduling is an important component of the training recipe because it can influence convergence speed, final accuracy, and generalization. However, scheduler selection is often treated as a secondary hyperparameter and many studies report results using only one fixed scheduler.

The specific objective is therefore to systematically compare different learning-rate scheduling strategies while testing them across heterogeneous neural-network architectures.

The study evaluates:

- 30 neural-network architectures;
- 25 scheduler configurations;
- 9 scheduler families;
- 3,938 evaluated model variants;
- CIFAR-10 classification;
- top-1 classification accuracy as the primary metric.

The study is particularly relevant to our debate because it allows us to ask not simply whether cosine performs well, but whether its performance advantage over StepLR remains consistent across different architectures.

3.7.3. Research Methodology

The methodology combines **theoretical optimization analysis with extensive empirical experimentation**.

Theoretical approach

The authors formulate learning-rate selection as a **min-max optimization problem**.

The goal is to find learning-rate values that minimize the worst-case loss within a fixed number of training iterations.

The framework considers changes in the loss landscape caused by:

- dataset characteristics,
- sampling,
- network architecture,
- parameter configuration,
- optimization conditions.

The authors approximate the local loss landscape using the **Hessian matrix** and derive a learning-rate function from this optimization formulation.

The proposed UBA schedule is controlled by a single parameter, **φ**, which controls the variation speed of the learning rate.

An important theoretical result is that when φ approaches 2, the proposed UBA schedule becomes equivalent to the standard **cosine learning-rate schedule**.

This is significant for our debate because the paper does not simply treat cosine decay as an arbitrary heuristic. Instead, cosine appears as a **special case within a theoretically derived scheduling framework**.

Experimental Evaluation

The researchers compared UBA with:

- Step (SS)
- Cosine (CS)
- Cyclical Learning Rate (CLR)
- OneCycle (1C)
- Budgeted Training (BT)
- REX

The evaluation covered:

Vision

- CIFAR-10
- CIFAR-100
- ImageNet

using:

- ResNet-18

- ResNet-34
- ResNet-50

The models were independently trained using:

- 25% training budget
- 50% training budget
- 100% training budget.

Architecture Pool

The researchers selected **30 architectures** from the LEMUR neural-network dataset, covering convolutional, mobile-optimized, dense, and transformer-based models.

The architecture pool includes models such as:

- ResNet
- VGG
- DenseNet
- EfficientNet
- MobileNetV2/V3
- GoogLeNet
- InceptionV3
- SwinTransformer
- VisionTransformer
- ConvNeXt
- BagNet
- FractalNet

Scheduler Configurations

Family	Key Parameters	#
StepLR	step$\in\{0.1,0.25,0.5,0.7\}$, $\gamma \in\{0.1,0.3,0.5,0.7\}$	4
ExponentialLR	$\gamma \in\{0.90,0.95,0.98\}$	3
CosineAnnealingLR	$T_{\max} \in\{5,10,20\}$ epochs	3
CosineAnnealingWR	$T_0 \in\{2,5\}$	2
MultiStepLR	milestones $\{5,10\},\{3,7\},\{2,4\}$	3
ReduceLROnPlateau	(factor,patience): (0.5,2),(0.3,3),(0.1,5)	3
CyclicLR	triangular, triangular2, exp_range	3
OneCycleLR	$\mathrm{lr}_{\max} \in\{0.05,0.10\}$	2
LinearLR / PolynomialLR	fixed defaults	2
Total		**25**

Summary of the 25 learning-rate scheduler configurations evaluated in the study.

Experimental Setup

All variants were evaluated on CIFAR-10.

The models were trained using:

- SGD
- learning rate = 0.01
- momentum = 0.9
- batch size = 64
- dropout = 0.2
- five training epochs
- seven weight-decay values.

The theoretical search space was:

30 architectures × 25 scheduler configurations × 7 weight-decay values = 5,250 possible variants.

After validation and filtering, 3,938 variants were evaluated.

The primary metric was top-1 accuracy. The authors additionally examined mean accuracy, median accuracy, high-performing variants, scheduler-family performance, and architecture-specific scheduler preferences.

3.7.4. Key Findings and Evidence Extracted

Finding 1: Cosine-based scheduling achieved higher overall mean accuracy than StepLR

The large-scale evaluation demonstrates that learning-rate scheduler selection has a measurable effect on neural-network performance. The study evaluated **3,938 model variants** on CIFAR-10 and compared multiple learning-rate scheduler families under a unified experimental framework. Among the evaluated schedulers, **CosineAnnealingLR achieved a mean top-1 accuracy of 54.55%, while StepLR achieved 53.70%**. This corresponds to a **0.85 percentage-point difference** in mean accuracy.

The results also show that CosineAnnealingLR achieved a best individual accuracy of 85.99%, compared with 85.55% for StepLR. Although the difference between the two schedulers is smaller than the differences observed between some other scheduler families, the result demonstrates that

CosineAnnealingLR was associated with stronger average performance within the evaluated model-variant pool.

Scheduler Family	Mean Acc. (%)	Best Acc. (%)
CosineAnnealingWR	**55.20**	**86.45**
MultiStepLR	54.65	84.74
CosineAnnealingLR	54.55	85.99
ExponentialLR	54.51	85.96
PolynomialLR	54.11	85.12
StepLR	53.70	85.55
OneCycleLR	53.67	86.17
LinearLR	50.05	84.53
CyclicLR	49.81	84.25
ReduceLROnPlateau	45.04	**86.45**

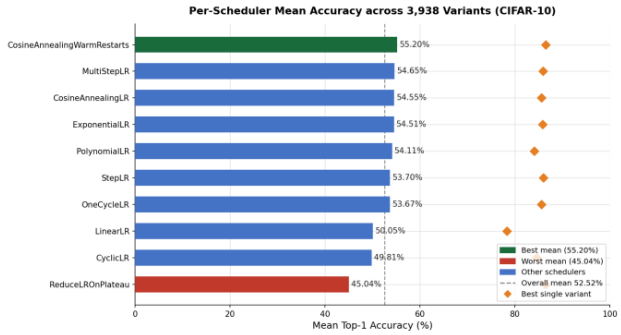
Table 1 - Mean and best top-1 accuracy per scheduler family.

Scheduler	Mean Accuracy	Best Accuracy
CosineAnnealingLR	54.55%	85.99%
StepLR	53.70%	85.55%

Evidence interpretation

Table 1 shows that CosineAnnealingLR produced a higher mean accuracy than StepLR across the evaluated model variants. The comparison is based on the mean accuracy, which provides an indication of the overall performance of the scheduler family rather than focusing only on its single best-performing configuration.

The results also show that CosineAnnealingWarmRestarts obtained the highest mean accuracy at 55.20%, while MultiStepLR achieved 54.65% and CosineAnnealingLR achieved 54.55%. Therefore, the findings indicate that several smooth or milestone-based learning-rate schedules performed strongly under the experimental conditions.



Mean top-1 accuracy per scheduler family across all 3,938 evaluated variants on CIFAR-10.

The figure provides a visual representation of the scheduler-family comparison. The horizontal bars represent mean top-1 accuracy, while the markers indicate the best individual result obtained by each scheduler family.

The visualisation makes the relative performance of CosineAnnealingLR and StepLR apparent. CosineAnnealingLR reaches **54.55% mean accuracy**, compared with **53.70% for StepLR**.

An important observation from the figure is that **mean performance and peak performance are different measures**. For example, ReduceLROnPlateau has the lowest mean accuracy at **45.04%**, but its best individual configuration reaches **86.45%**. This demonstrates why both aggregate and individual performance should be considered when evaluating learning-rate scheduling strategies.

Finding 2: The strongest CosineAnnealingLR configuration achieved higher mean accuracy than the strongest StepLR configuration

The study also examines individual scheduler configurations to determine how parameter choices influence performance within each scheduler family.

Among the evaluated StepLR configurations, StepLR_s20_g07 achieved the highest mean accuracy of 62.79%. In comparison, CosineAnnealingLR_T10 achieved a mean accuracy of 63.97%, making it the strongest reported CosineAnnealingLR configuration. The difference between these two configurations is 1.18 percentage points.

Model ID	Top-1 Accuracy (%)
lr-baf515f1...	**86.45**
lr_2782	86.17
lr_2773	85.99
lr-982834f1...	85.96
lr_2777	85.69
lr_2784	85.56
lr_2797	85.55
lr_2794	85.55
lr_2779	85.52
lr-afd558c8...	85.49

Table 2 - Per-variant accuracy for selected scheduler configurations on CIFAR-10.

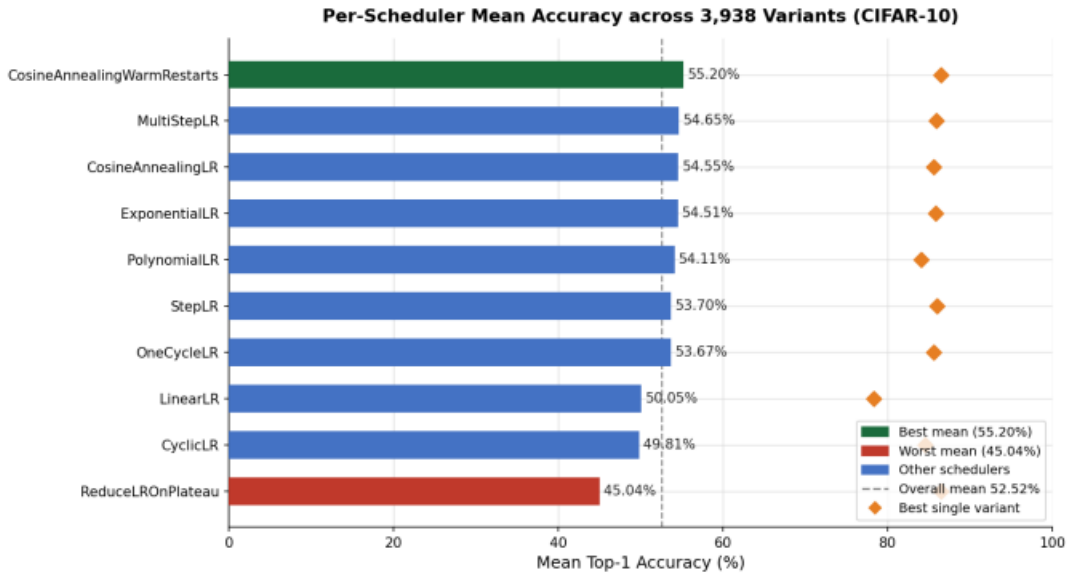
Variant	Mean (%)	Best (%)
StepLR_s20_g07	62.79	82.99
StepLR_s3_g01	58.55	81.30
StepLR_s10_g05	57.33	82.35
StepLR_s5_g03	56.34	82.44
ExponentialLR_g098	59.98	84.74
ExponentialLR_g09	58.92	85.41
ExponentialLR_g095	54.92	84.79
CosineAnnealingLR_T10	63.97	82.55
CosineAnnealingLR_T5	61.43	83.67
CosineAnnealingLR_T20	58.48	84.29

Table

Evidence interpretation

Table X shows that CosineAnnealingLR produced a higher mean accuracy than StepLR across the evaluated model variants. The comparison is based on the **mean accuracy**, which provides an indication of the overall performance of the scheduler family rather than focusing only on its single best-performing configuration.

The results also show that CosineAnnealingWarmRestarts obtained the highest mean accuracy at **55.20%**, while MultiStepLR achieved **54.65%** and CosineAnnealingLR achieved **54.55%**. Therefore, the findings indicate that several smooth or milestone-based learning-rate schedules performed strongly under the experimental conditions.



Mean top-1 accuracy per scheduler family across all 3,938 evaluated variants on CIFAR-10

The figure provides a visual representation of the scheduler-family comparison. The horizontal bars represent mean top-1 accuracy, while the markers indicate the best individual result obtained by each scheduler family.

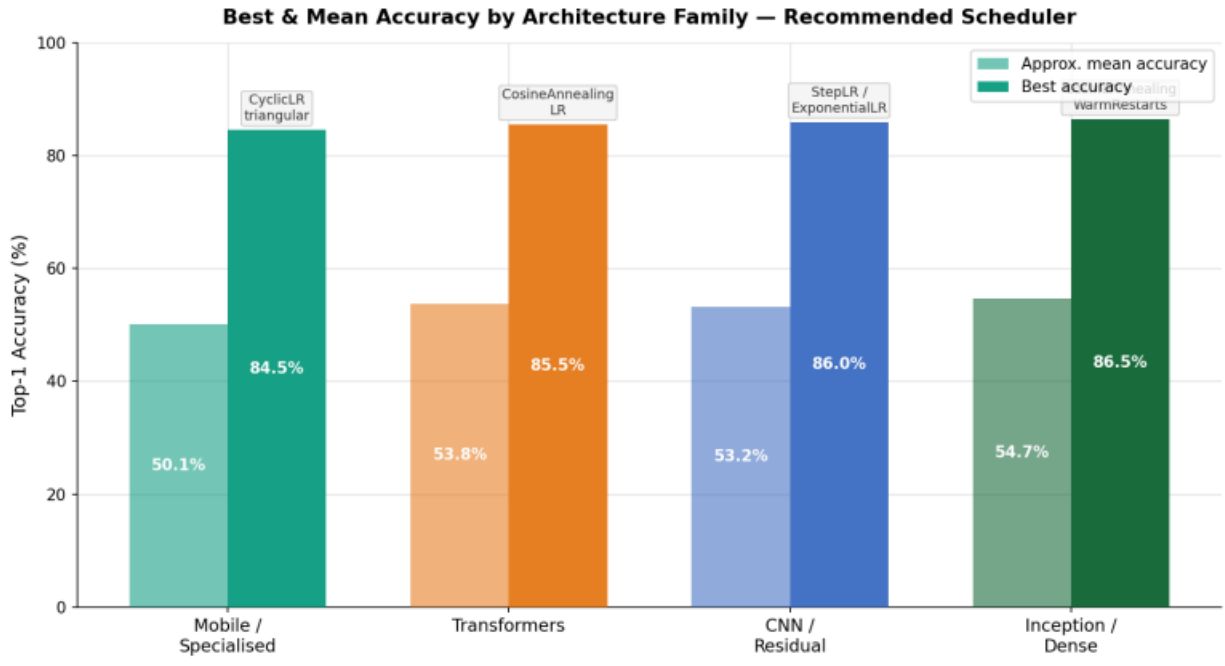
The visualisation makes the relative performance of CosineAnnealingLR and StepLR apparent. CosineAnnealingLR reaches **54.55% mean accuracy**, compared with **53.70% for StepLR**.

An important observation from the figure is that **mean performance and peak performance are different measures**. For example, ReduceLROnPlateau has the lowest mean accuracy at **45.04%**, but its best individual configuration reaches **86.45%**. This demonstrates why both aggregate and individual performance should be considered when evaluating learning-rate scheduling strategies.

Finding 3: Scheduler effectiveness varied substantially across neural-network architectures

The architecture-level analysis demonstrates that the performance of a learning-rate scheduler is not consistent across all neural-network architectures.

The study evaluated architectures from different design families, including convolutional, mobile-oriented, dense-connection and transformer-based models. The best scheduler differed between these architectures. For example, SwinTransformer, MobileNetV2 and ConvNeXt achieved their highest reported accuracies using CosineAnnealingLR, whereas FractalNet, EfficientNet and BagNet achieved their highest reported accuracies using StepLR.



Architecture	Best Accuracy (%)	Best Scheduler
SwinTransformer	85.46	CosineAnnealingLR
MobileNetV2	84.55	CosineAnnealingLR
ConvNeXt	83.73	CosineAnnealingLR
FractalNet	85.55	StepLR
EfficientNet	85.12	StepLR
BagNet	84.69	StepLR

Best and approximate mean top-1 accuracy by architecture family on CIFAR-10.

The figure compares approximate mean and best accuracy across architecture families and identifies the scheduler associated with the strongest result for each family.

Metric	Value
Total variants contributed & evaluated	3,938
Maximum accuracy	86.45%
Mean accuracy	52.52%
Median accuracy	50.00%
Minimum accuracy	12.08%
Variants > 80% accuracy	237 (6.0%)
Variants > 75% accuracy	627 (15.9%)
Variants > 60% accuracy	1,107 (28.1%)
Variants < 30% accuracy	445 (11.3%)

Best top-1 accuracy and corresponding scheduler for selected neural-network architectures.

Evidence interpretation

The architecture-level results demonstrate that the effect of a learning-rate scheduler is dependent on the characteristics of the network being trained.

CosineAnnealingLR is associated with the best reported results for several architectures, including **SwinTransformer, MobileNetV2 and ConvNeXt**. However, StepLR produces the highest reported result for **FractalNet, EfficientNet and BagNet**.

The study therefore identifies a clear architecture–scheduler interaction. The authors explicitly report that **no single scheduler is universally optimal across architecture families**. Transformer-based models favour CosineAnnealingLR in the reported results, whereas mobile and convolutional models can favour other scheduling strategies.

Finding 4: Scheduler performance was influenced by the available training budget

The experiments were conducted using a **five-epoch screening budget**. This limited training period is important because different learning-rate schedules change the learning rate at different points during training.

The configuration-level results show that the timing of a learning-rate reduction can substantially affect performance. Within the StepLR family, the **s20_g07 configuration achieved 62.79% mean accuracy**, while the earlier-decay configurations achieved lower mean accuracies:

- s20_g07 → **62.79%**
- s3_g01 → **58.55%**
- s10_g05 → **57.33%**
- s5_g03 → **56.34%**.

The difference between the strongest and weakest reported StepLR variants is:

62.79 − 56.34 = 6.45 percentage points.

The study interprets this result as indicating that, within the five-epoch setting, preserving a relatively high learning rate for a larger portion of the training budget can be beneficial.

The paper also notes that some schedules depend on longer training periods to exhibit their intended behaviour. For example, the average performance of ReduceLROnPlateau is affected because five epochs may not provide sufficient time for meaningful plateau detection.

Interpretation

These results demonstrate that scheduler performance should be considered in relation to the **training horizon**. A schedule that performs well under a long training process may not necessarily behave similarly when the available number of training epochs is substantially reduced.

Finding 5: No single scheduler consistently produced the best result across all architectures

The complete architecture–scheduler analysis reveals that there is no universally optimal learning-rate scheduler.

The study reports that **mobile and convolutional models**, including MobileNetV2/V3, EfficientNet and ResNet, can strongly prefer CyclicLR, while **GoogLeNet, BagNet and InceptionV3** achieve peak performance using CosineAnnealingWarmRestarts or ReduceLROnPlateau. Transformer-based models such as **SwinTransformer and VisionTransformer favour CosineAnnealingLR** in the reported results.

This finding is important because it demonstrates that scheduler selection is not simply a matter of selecting the scheduler with the highest overall mean.

The results instead suggest that scheduler performance emerges from the interaction between:

- scheduler design,

- scheduler parameters,
- architecture,
- training budget, and
- other training conditions.

The authors consequently recommend architecture-specific scheduler selection rather than relying on a single global prescription

3.7.5. Relevance to the Debate

This paper is a particularly valuable source for the affirmative case because it provides a **large-scale, systematic empirical comparison** of CosineAnnealingLR and StepLR rather than relying on a single model, dataset configuration, or isolated experiment.

● **It provides direct empirical evidence comparing the two original scheduler families.** The study evaluates both **CosineAnnealingLR and StepLR** within the same experimental framework, using **3,938 evaluated model variants across 30 neural-network architectures on CIFAR-10**. This makes the comparison substantially broader than an experiment involving only one architecture or one carefully selected configuration.

● **The scheduler-family results directly favour CosineAnnealingLR in mean performance.** Across the evaluated variants, CosineAnnealingLR achieved a mean top-1 accuracy of **54.55%**, compared with **53.70% for StepLR**, giving CosineAnnealingLR a **0.85 percentage-point advantage**. The best individual result was also slightly higher for CosineAnnealingLR (**85.99%**) than for StepLR (**85.55%**). This provides a direct numerical comparison between the two scheduling approaches.

● **The individual-configuration analysis strengthens the comparison rather than relying only on family averages.** The strongest reported CosineAnnealingLR configuration, **T10**, achieved a mean accuracy of **63.97%**, while the strongest reported StepLR configuration, **s20 g07**, achieved **62.79%**. This gives CosineAnnealingLR a further **1.18 percentage-point advantage** at the configuration level.

● **The study provides evidence across heterogeneous architectures, making the findings more informative than a single benchmark comparison.** CosineAnnealingLR achieved the best reported results for architectures including **SwinTransformer, MobileNetV2 and ConvNeXt**, demonstrating that its strong performance is observed across different neural-network designs.

● **A fair-minded limitation must also be recognised.** The paper does not demonstrate that CosineAnnealingLR is universally superior to StepLR. StepLR achieved the best reported results for **FractalNet, EfficientNet and BagNet**, while other architectures preferred other scheduler families. In addition, the experiments use a **five-epoch training budget**, fixed SGD settings, and the CIFAR-10 dataset. Therefore, the strongest interpretation of the evidence is not that cosine decay always wins, but that **CosineAnnealingLR demonstrates stronger overall and configuration-level mean performance within this systematic evaluation, while its effectiveness remains dependent on architecture and training conditions**.

3.7.6. Position Supported by the Paper

Affirmative / Positive.

The paper **supports the affirmative position** through direct empirical evidence showing that CosineAnnealingLR achieves higher mean performance than StepLR in the overall evaluation. Across **3,938 model variants**, CosineAnnealingLR achieved **54.55% mean top-1 accuracy**, compared with **53.70% for StepLR**. At the configuration level, the strongest reported CosineAnnealingLR configuration (**T10: 63.97%**) also exceeded the strongest reported StepLR configuration (**s20 g07: 62.79%**).

However, the paper's evidence supports a **qualified rather than absolute interpretation**. The architecture-level analysis shows that scheduler performance varies across neural-network architectures, with StepLR achieving the best result for several architectures. The study therefore provides strong evidence that cosine decay can outperform step decay in aggregate and under selected configurations, but it does not establish universal superiority across every architecture or training condition.

Overall, this paper supports the affirmative position primarily on the basis of large-scale empirical performance: CosineAnnealingLR achieves higher overall mean accuracy and a higher best-performing configuration than StepLR within the study's controlled evaluation, while the architecture-dependent results provide an important qualification to the strength of that conclusion.

3.8. Research Paper 8

3.8.1. Full Reference

A.Tang, Y. Dong, Y. Zeng, X. Zhou, and Z. Lin, **"Stepsize Anything: A Unified Learning Rate Schedule for Budgeted-Iteration Training,"** *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS)*, 2025.

3.8.2. Research Objective

Modern deep-learning models require increasingly large computational resources for training. As model size, dataset size and training costs increase, practitioners may have to train models under a **fixed computational or iteration budget**. The authors identify this situation as **budgeted-iteration training**, where the objective is to achieve the best possible model performance within a predetermined number of training iterations.

The researchers identify two important problems with existing learning-rate scheduling approaches. First, existing schedules are often designed using heuristic rules rather than a strong theoretical foundation. Second, selecting an appropriate schedule can require extensive trial-and-error experimentation, which becomes particularly expensive when computational resources are limited.

To address these issues, the paper proposes a **Unified Budget-Aware (UBA)** learning-rate schedule. The objective is to develop a schedule that can adapt to different training budgets while providing a theoretically motivated mechanism for controlling learning-rate behaviour.

The proposed schedule is governed primarily by a single parameter, **φ**, which the authors connect theoretically to the **condition number of the local optimization landscape**. The paper establishes convergence results for different values of φ and investigates the behaviour of the proposed method experimentally across different datasets, architectures, training budgets and optimizers.

Central research objectives

The study therefore aims to:

1. Develop a theoretically grounded learning-rate schedule for fixed training budgets.
2. Establish a mathematical relationship between the schedule parameter φ and optimization difficulty.
3. Investigate whether the proposed schedule can adapt to different training budgets.
4. Compare UBA against commonly used schedules, including **Step and Cosine**.
5. Evaluate scheduling behaviour across different datasets and neural-network architectures.
6. Examine the effect of different optimizers and learning-rate configurations.

3.8.3. Research Methodology

Learning-Rate Scheduling Methods Compared

The study evaluates UBA against several established learning-rate scheduling approaches.

The baselines include:

- **Step Schedule (SS)**
- **Cosine Schedule (CS)**
- **Cyclical Learning Rate (CLR)**
- **OneCycle (1C)**
- **Budgeted Training (BT)**
- **Reflected Exponential (REX)**

The paper defines Step scheduling as a schedule involving discrete reductions in the learning rate. For the experiments, the Step schedule reduces the learning rate by **×0.1 after 50% of training and ×0.01 after 75% of training**. The authors note that Step scheduling requires careful tuning of the timing of these abrupt changes.

Cosine scheduling is implemented using a smooth cosine-shaped reduction from the initial learning rate toward the minimum learning rate. The authors describe its smooth transition as a way of avoiding abrupt learning-rate decreases.

Dataset	Network	Method	training budget (epoch(%))		
			75 (25%)	150(50%)	300(100%)
CIFAR10	ResNet18	SS	92.71±0.4243	93.45±0.4822	93.61±0.3035
		CS	94.21±0.5116	94.24±0.2419	95.52±0.2433
		CLR	92.13±0.2610	92.99±0.5459	94.92±0.3811
		1C	94.23±0.1838	95.03±0.1353	95.35±0.2984
		BT	94.18±0.3546	94.79±0.4605	**95.68±0.1168**
		REX	94.49±0.3225	95.10±0.1929	95.45±0.1350
		UBA(ours)	**94.57±0.3281**	**95.26±0.2150**	95.65±0.1589
			75 (25%)	150(50%)	300(100%)
CIFAR100	ResNet34	SS	71.09±0.2676	75.34±0.1802	77.24±0.1808
		CS	64.12±0.1808	69.84±0.7580	74.29±0.5958
		CLR	73.33±0.5635	73.41±0.2835	74.74±0.2266
		1C	73.38±0.3227	75.04±0.6382	77.86±0.2239
		BT	72.75±0.2611	75.35±0.2423	78.58±0.2705
		REX	73.64±0.5635	75.14±0.3110	78.04±0.1430
		UBA(ours)	**74.60±0.1508**	**76.63±0.2678**	**78.95±0.2818**
			75 (25%)	150(50%)	300(100%)
ImageNet	ResNet50	SS	74.74±0.1386	77.24±0.3861	78.56±0.6435
		CS	75.84±0.2220	77.95±0.2164	79.07±0.0438
		CLR	73.90±1.4609	76.04±1.6419	77.84±1.3053
		1C	75.83±0.7792	77.74±0.5218	78.90±0.2956
		BT	75.90±0.2786	77.81±0.4624	78.86±0.2814
		REX	75.85±0.8047	77.66±0.8853	78.66 ±0.2828
		UBA(ours)	**76.29±0.4031**	**78.26±0.1640**	**79.39±0.0170**

Learning-rate scheduling methods evaluated in the study.

The original table demonstrates that UBA can reproduce the behaviour of several established schedules through parameter adjustments, including:

- Cosine → φ = 2
- Step → φ = 0 with corresponding step configuration
- Exponential → φ = 30
- Cyclic → φ = 2 with cyclic interval
- REX → φ = 0.8
- OneCycle → φ = 2 with appropriate cycle settings.

Why this table is useful

This table establishes that the paper does not treat Cosine and Step as unrelated scheduling mechanisms. Instead, the proposed UBA formulation provides a unified mathematical framework capable of representing several existing learning-rate behaviors.

Theoretical Formulation of UBA

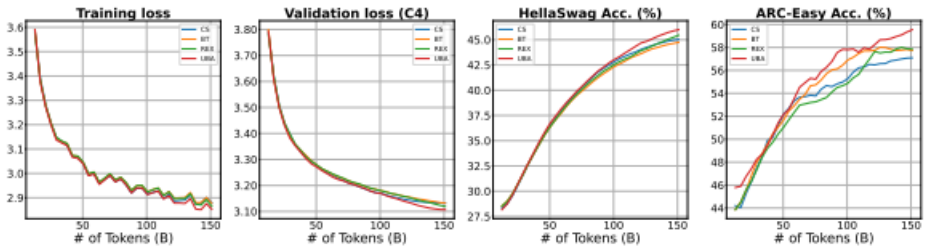
The proposed UBA schedule is derived from an optimization framework that explicitly considers the behaviour of the loss landscape.

The authors establish a relationship between the parameter **φ** and the condition number of the local loss landscape. According to their theoretical formulation:

- smaller φ values are associated with better-conditioned optimization problems;
- larger φ values are associated with more difficult or poorly conditioned optimization problems.

The authors further demonstrate that when **φ approaches 2**, the proposed schedule reduces to the standard cosine learning-rate schedule.

.



The figure illustrates how changing φ changes the shape and behaviour of the learning-rate schedule.

The theoretical analysis indicates that when φ is close to 2, the schedule naturally approaches the cosine form. The authors also demonstrate that the UBA formulation can approximate other established scheduling strategies through suitable parameter adjustments.

Interpretation

This theoretical component provides a mathematical explanation for why the proposed schedule can produce different learning-rate trajectories rather than treating the learning-rate curve purely as an empirically selected heuristic.

Vision Experimental Design

The main empirical evaluation focuses on image-classification tasks.

The researchers evaluate:

Dataset	**Architecture**
CIFAR-10	ResNet18
CIFAR-100	ResNet34
ImageNet	ResNet50

The experiments are conducted using different training budgets corresponding to **25%, 50% and 100%** of the maximum training budget. Importantly, the models are **independently trained at each budget**, rather than obtaining the shorter-budget results by truncating or interpolating results from longer training runs.

The CIFAR experiments use:

- SGD optimizer
- initial learning rate = **0.1**
- batch size = **128**
- weight decay = $\mathbf{5 \times 10^{-4}}$
- PyTorch implementation.

Each CIFAR epoch contains approximately **391 training steps**.

For ImageNet, the study uses **ResNet50** and approximately **1.28 million training images**, making it a substantially larger-scale evaluation than CIFAR.

Evaluation Metrics and Experimental Reliability

The primary performance measure for the vision experiments is **validation accuracy**.

The study reports mean values with variation estimates for the main vision comparisons. Several independent runs are conducted to obtain uncertainty estimates, while the ImageNet-ResNet50 experiments use two independent runs because of computational constraints.

The experimental design therefore allows the researchers to compare scheduling strategies under:

- different datasets,
- different model architectures,
- different training budgets,
- different optimizers, and
- different initial learning rates.

This multidimensional evaluation is important because scheduler performance can depend on the wider training configuration.

3.8.4. Key Findings and Evidence Extracted

Finding 1: Cosine scheduling consistently achieved higher validation accuracy than Step scheduling on CIFAR-10 across all evaluated training budgets

The first direct comparison between Cosine and Step is obtained from the **CIFAR-10/ResNet18** experiment.

Relevant CIFAR-10 results

Training Budget	Step (SS)	Cosine (CS)	Cosine Advantage
25%	92.71 ± 0.4243	**94.21 ± 0.5116**	**+1.50 pp**
50%	93.45 ± 0.4822	**94.24 ± 0.2419**	**+0.79 pp**
100%	93.61 ± 0.3035	**95.52 ± 0.2433**	**+1.91 pp**

Evidence interpretation

Cosine achieves higher validation accuracy than Step at **all three training budgets**.

At the 25% budget, Cosine achieves **94.21%**, compared with **92.71%** for Step. At 50%, the corresponding values are **94.24% and 93.45%**, while at the full budget Cosine reaches **95.52% compared with 93.61% for Step**.

The advantage therefore remains visible across short, intermediate and full training budgets. The largest observed difference occurs at the 100% budget, where Cosine exceeds Step by **1.91 percentage points**.

This provides consistent evidence that, under the CIFAR-10/ResNet18 experimental setting, Cosine scheduling produces higher validation accuracy than the Step schedule used in the study.

Finding 2: Cosine scheduling maintained higher validation accuracy than Step scheduling on the larger-scale ImageNet benchmark

To examine whether the scheduling behaviour extends beyond CIFAR, the researchers evaluate **ImageNet using ResNet50**.

Training Budget	Step (SS)	Cosine (CS)	Cosine Advantage
25%	74.74 ± 0.1386	**75.84 ± 0.2220**	**+1.10 pp**
50%	77.24 ± 0.3861	**77.95 ± 0.2164**	**+0.71 pp**
100%	78.56 ± 0.6435	**79.07 ± 0.0438**	**+0.51 pp**

Evidence interpretation

Cosine again achieves higher validation accuracy than Step at every evaluated training budget.

The difference is **1.10 percentage points at 25%**, decreases to **0.71 percentage points at 50%**, and becomes **0.51 percentage points at 100%**.

Although the magnitude of the difference becomes smaller as the training budget increases, Cosine maintains the higher validation accuracy at all three evaluated budgets.

This experiment is particularly informative because ImageNet contains approximately **1.28 million training images across 1,000 classes**, providing a substantially larger-scale test environment than CIFAR.

Finding 3: Step scheduling substantially outperformed Cosine scheduling on CIFAR-100

The CIFAR-100/ResNet34 experiment produces a substantially different result from CIFAR-10 and ImageNet.

Training Budget	Step (SS)	Cosine (CS)	Step Advantage
25%	**71.09 ± 0.2676**	64.12 ± 0.1808	**+6.97 pp**
50%	**75.34 ± 0.1802**	69.84 ± 0.7580	**+5.50 pp**
100%	**77.24 ± 0.1808**	74.29 ± 0.5958	**+2.95 pp**

Source: Paper 02, Table 2

Evidence interpretation

In contrast to the CIFAR-10 and ImageNet experiments, Step scheduling achieves substantially higher validation accuracy than Cosine across **all three training budgets**.

At the 25% budget, Step achieves **71.09%**, while Cosine achieves only **64.12%**, resulting in a difference of **6.97 percentage points**.

At 50%, Step maintains a **5.50 percentage-point advantage**, while at the full training budget the difference remains **2.95 percentage points**.

This is one of the most important findings in the study because it demonstrates that the relative performance of learning-rate schedules can change substantially according to the **dataset and architecture combination**.

Finding 4: The relative performance of Cosine and Step changed when the initial learning rate was changed

The researchers conduct an additional experiment specifically to investigate how the initial learning rate influences scheduling performance.

The experiment uses **CIFAR-100 with ResNet34** and evaluates:

- LR = 0.1
- LR = 0.01
- LR = 0.001.

The auxiliary experiment uses **three independent runs** and slightly fewer epochs than the main experiments because of computational efficiency.

Schedule	LR = 0.1	LR = 0.01	LR = 0.001
Step	**77.28 ± 0.2401**	73.48 ± 0.4267	67.98 ± 0.1473

Cosine	74.11 ± 0.7690	**74.66 ± 0.5888**	**69.18 ± 0.3736**

Source: Paper 02, Table 11.

Evidence interpretation

The comparison changes according to the initial learning rate.

At **LR = 0.1**, Step achieves **77.28%**, exceeding Cosine's 74.11%.

However, at **LR = 0.01**, Cosine achieves **74.66%**, compared with Step's 73.48%.

At **LR = 0.001**, Cosine again performs better, achieving **69.18%**, compared with Step's 67.98%.

This demonstrates that the relative performance of the two schedules is not independent of the initial learning-rate setting.

Finding 5: The behaviour of the proposed scheduling framework varied with the optimizer

The researchers also examine the proposed scheduling framework under different optimizers.

The main vision experiments use **SGD**, while the cross-optimizer analysis additionally evaluates **AdamW** on CIFAR-100 and ImageNet. For AdamW, the paper uses a learning rate of **0.003** and weight decay of **0.0001**.

Dataset -network	Optimizer	Schedule	training budget (epoch(%))		
			30 (25%)	60(50%)	120(100%)
CIFAR100 -ResNet34	SGD	SS	70.85	*75.58*	77.28
		CS	63.88	68.84	70.43
		CLR	72.63	73.81	74.80
		1C	*73.72*	75.53	77.90
		BT	72.53	75.40	78.49
		REX	73.12	75.48	*77.99*
		UBA(ours)$\varphi = 5$	**74.57**	**76.68**	**78.97**
	AdamW	SS	64.01	70.93	73.65
		CS	62.56	68.91	72.98
		CLR	62.87	70.13	73.22
		1C	64.27	71.60	74.06
		BT	64.01	71.44	*74.34*
		REX	**65.19**	*71.72*	74.08
		UBA(ours)$\varphi = 0.5$	*64.44*	**72.10**	**74.48**
			75 (25%)	150(50%)	300(100%)
ImageNet -ResNet50	SGD	SS	74.84	76.96	78.10
		CS	*75.99*	*77.79*	*79.10*
		CLR	72.86	74.88	76.91
		1C	75.28	77.37	78.79
		BT	75.71	77.48	78.66
		REX	75.28	77.03	78.46
		UBA(ours)$\varphi = 5$	**76.00**	**77.99**	**79.32**
	AdamW	SS	74.64	77.51	79.01
		CS	75.68	78.10	79.04
		CLR	74.93	77.20	78.76
		1C	76.38	78.11	79.21
		BT	76.10	78.14	*79.05*
		REX	*76.42*	*78.28*	78.86
		UBA(ours) $\varphi = 0.5$	**76.57**	**78.37**	**79.38**

The cross-optimizer experiment shows that the proposed UBA framework remains effective under both SGD and AdamW. However, the optimal behaviour of the UBA parameter φ differs between optimizers.

Evidence interpretation

The paper explains this difference using the characteristics of the optimizers. AdamW introduces adaptive mechanisms that can reduce the effects of ill-conditioning, while SGD does not provide the same form of adaptive preconditioning. Consequently, the authors report that **smaller φ values are generally favoured with AdamW, whereas larger φ values are more suitable with SGD**.

This provides evidence that learning-rate scheduling behaviour interacts with the underlying optimization algorithm rather than functioning independently from it.

Finding 7: The preferred UBA parameter changed according to optimization difficulty and optimizer

The paper performs a sensitivity analysis using several values of φ:

0.25, 0.5, 1, 2.5, 5 and 10.

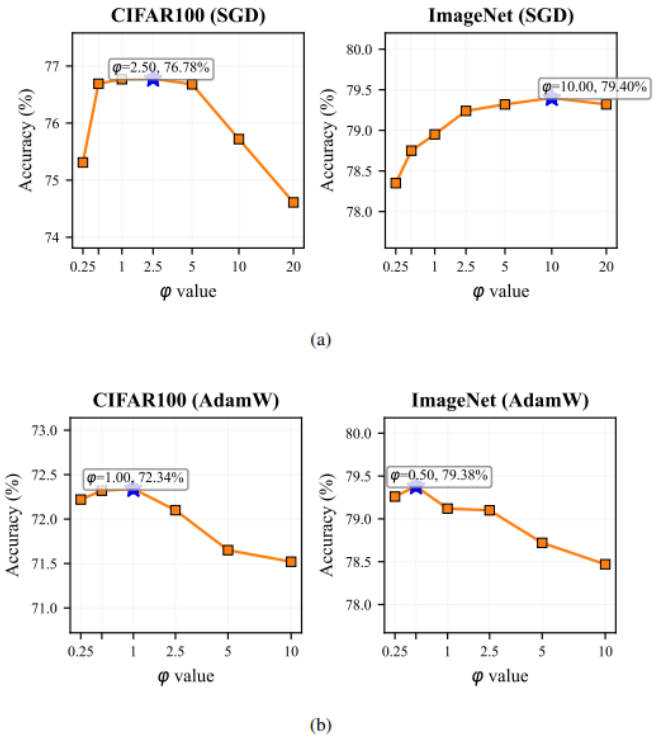


Figure 11: Visualization of cross-φ performance.

Dataset	Optimizer	φ = 0.25	φ = 0.5	φ = 1	φ = 2.5	φ = 5	φ = 10
CIFAR-100 / ResNet34	SGD	75.10	75.89	76.31	76.40	**76.68**	75.64
CIFAR-100 / ResNet34	AdamW	**72.10**	71.34	71.54	71.21	70.40	69.01
ImageNet / ResNet50	SGD	78.35	78.75	78.95	79.24	79.32	**79.40**
ImageNet / ResNet50	AdamW	79.26	**79.38**	79.12	79.10	78.72	78.47

Source: Paper 02, Table 10.

Evidence interpretation

The results show that the preferred φ value changes depending on both the optimizer and dataset.

For example, on CIFAR-100 with SGD, the highest reported result among the tested values occurs at **φ = 5**, whereas AdamW performs best at **φ = 0.25**. On ImageNet, SGD performs best at **φ = 10**, while AdamW performs best at **φ = 0.5**.

The authors relate this behaviour to optimization difficulty and the preconditioning properties of the optimizer. This provides additional evidence that the shape and behaviour of a learning-rate schedule are influenced by the characteristics of the optimization process.

3.8.5. Relevance to the Debate

This paper is a particularly valuable source for the affirmative case because it provides **recent, direct and multi-condition experimental evidence** involving the two original scheduling approaches under consideration: **Cosine and Step**.

● **It provides direct Cosine-versus-Step comparisons across multiple training budgets.** The vision experiments independently evaluate 25%, 50% and 100% training budgets, allowing the relative behaviour of the two schedules to be examined under different amounts of available training. On CIFAR-10/ResNet18, Cosine achieves higher validation accuracy than Step at all three budgets.

● **The evidence extends beyond a single small benchmark.** The study evaluates CIFAR-10 with ResNet18, CIFAR-100 with ResNet34, and ImageNet with ResNet50. Cosine outperforms Step across all three ImageNet training budgets, providing evidence from a substantially larger-scale image-classification benchmark.

● **The paper provides particularly strong evidence that the relative performance of the schedules depends on experimental conditions.** CIFAR-100 produces the opposite result from CIFAR-10 and

ImageNet: Step outperforms Cosine by **6.97, 5.50 and 2.95 percentage points** at the 25%, 50% and 100% training budgets respectively. This makes the paper valuable not only because of the positive results for Cosine, but because it provides a transparent demonstration of when Step scheduling can be stronger.

● **The initial-learning-rate experiment provides an additional layer of evidence.** At an initial learning rate of **0.1**, Step achieves 77.28% compared with Cosine's 74.11%, while at 0.01 and 0.001 Cosine achieves higher validation accuracy than Step. This demonstrates that scheduler performance can change when the initial learning rate changes.

● **A fair-minded qualification is essential.** The paper's primary contribution is the proposed **UBA schedule**, not a dedicated study designed exclusively to establish Cosine superiority over Step. In fact, the paper reports that UBA itself can outperform both established schedules in several experiments, and the relative ranking of Step and Cosine changes across datasets and configurations. Therefore, the strongest interpretation of this source is not "Cosine universally dominates Step," but that **Cosine demonstrates consistent advantages in several important experimental settings while its relative performance remains dependent on dataset, architecture, training budget and initial learning rate**.

3.8.6. Position Supported by the Paper

Affirmative / Positive, with significant qualification.

The paper provides substantial empirical support for the affirmative position through its direct comparisons of **Cosine (CS)** and **Step (SS)**. On CIFAR-10 with ResNet18, Cosine achieves higher validation accuracy than Step at **25%, 50% and 100% training budgets**, with improvements of **1.50, 0.79 and 1.91 percentage points**, respectively.

The same pattern is observed on **ImageNet with ResNet50**, where Cosine achieves higher validation accuracy than Step at all three evaluated training budgets. The differences range from **0.51 to 1.10 percentage points**, demonstrating that Cosine maintains higher performance even in the larger-scale ImageNet setting.

However, the evidence also establishes an important limitation. On **CIFAR-100 with ResNet34**, Step substantially outperforms Cosine across all three training budgets, with the largest difference reaching **6.97 percentage points** at the 25% budget. Furthermore, changing the initial learning rate reverses the relative performance at some settings: Step is stronger at LR = 0.1, whereas Cosine is stronger at LR = 0.01 and 0.001.

Therefore, the paper supports the affirmative position **as a conditional empirical conclusion rather than a universal rule**.

Overall, this paper supports the affirmative position because Cosine scheduling outperforms Step scheduling across all evaluated training budgets on CIFAR-10/ResNet18 and ImageNet/ResNet50, while the initial-learning-rate experiments also show settings in which Cosine performs better. However, the substantial Step advantage on CIFAR-100/ResNet34 demonstrates that the relative effectiveness of the two schedules is dependent on the dataset, architecture, training budget and learning-rate configuration. The evidence therefore supports Cosine as a strong-performing scheduling strategy, but does not establish that it is universally superior to Step scheduling.

4. Claim–Evidence Matrix

4.1. Major Debate Claims and Supporting Evidence

The table below organizes the evidence from Section 3 by claim rather than by source. Each claim is backed by at least one paper, and several are backed by more than one — which matters if a judge or opponent challenges the reliability of any single study.

Claim	Evidence Type	Supporting Paper(s)	Key Data Point
1. Cosine decay has a genuine theoretical justification — it is not merely an empirical trick.	Theoretical	Eigencurve (2022); Li, Zhuang & Orabona (2021) [1]	First formal convergence proof for cosine decay on smooth non-convex functions (Li et al., Theorem 2); step decay carries a provable extra log(T) penalty that cosine-like schedules avoid (Eigencurve, Theorem 4).
2. Cosine decay is dramatically cheaper to tune than step decay.	Empirical / Practical	Li, Zhuang & Orabona (2021) [2]	Cosine decay needs only 1 hyperparameter to match or beat step decay, which needs 3–4 (milestone count, decay factor, per-milestone timing).
3. Cosine decay adapts automatically to gradient noise; step decay is not shown to do this.	Theoretical	Li, Zhuang & Orabona (2021) [2]	Under injected Gaussian noise, cosine and exponential step sizes kept converging with zero retuning, while every competing method (including step decay) plateaued at a noise-dependent error floor (Figure 5).
4. Cosine decay provides more consistent performance than traditional Step Decay across different training budgets and deep learning tasks.	Empirical / Practical	REX: Revisiting Budgeted Training with an Improved Schedule (2022) [3]	REX evaluates Step and Cosine schedules across multiple tasks and training budgets. Cosine achieves a higher proportion of top-performing results compared with Step Decay, showing stronger consistency across different settings.
5. Cosine decay improves practical deep	Empirical / Practical	He et al. (2019) – Bag of Tricks for	The paper shows that replacing step decay with cosine decay improves ImageNet

learning performance compared with traditional step decay by providing a smoother learning-rate transition.		Image Classification with Convolutional Neural Networks [4]	classification performance across multiple CNN architectures. Cosine decay achieved higher accuracy while providing smoother learning-rate adjustment compared with sudden step reductions.
6. CosineAnnealingLR demonstrates stronger overall empirical performance than StepLR across a large-scale evaluation of neural-network architectures and configurations.	Large-scale empirical comparison	*ystematic Evaluation of Learning Rate Scheduling Strategies Across Heterogeneous Architectures* [7]	The study evaluated **3,938 model variants across 30 neural-network architectures on CIFAR-10**. CosineAnnealingLR achieved a **54.55% mean top-1 accuracy**, compared with **53.70% for StepLR**, representing a **0.85 percentage-point advantage**. At the configuration level, the best reported CosineAnnealingLR configuration (T10) achieved **63.97% mean accuracy**, compared with **62.79%** for the strongest reported StepLR configuration (s20 g07), a **1.18 percentage-point advantage**.
7. Cosine scheduling can maintain an empirical advantage over Step scheduling across different training budgets and datasets, although its superiority is dependent on the experimental setting.	Multi-dataset and multi-budget empirical comparison	*Stepsize Anything: A Unified Learning Rate Schedule for Budgeted-Iteration Training* [8]	On **CIFAR-10/ResNet18**, Cosine outperformed Step at **25%, 50% and 100%** training budgets: **94.21% vs. 92.71%**, **94.24% vs. 93.45%**, and **95.52% vs. 93.61%**, respectively. On **ImageNet/ResNet50**, Cosine again outperformed Step at all three budgets: **75.84% vs. 74.74%**, **77.95% vs. 77.24%**, and **79.07% vs. 78.56%**. However, the same study found the opposite pattern on **CIFAR-100/ResNet34**, where Step exceeded Cosine by **6.97, 5.50 and 2.95 percentage points** at the three respective budgets.
8.Cosine decay provides a smoother learning-rate reduction than step decay, avoiding sudden changes in the learning rate.	Experimental / Mechanism-based evidence	Aach et al. (2023); Loshchilov & Hutter (2017)	Aach et al. explains that stepwise scheduling makes sharp learning-rate drops, while cosine annealing gradually decreases the learning rate throughout training. SGDR applies cosine annealing continuously during each training cycle.

9.Cosine decay can achieve better model performance than step decay under suitable training conditions.	Direct experimental evidence	Aach et al. (2023)	Cosine annealing achieved superior validation accuracy for small and medium batch sizes compared with stepwise annealing.
10.Cosine-based scheduling can improve training efficiency and convergence speed.	Experimental evidence	Loshchilov & Hutter (2017)	SGDR achieved comparable or better performance using 2×–4× fewer training epochs than commonly used learning-rate schedules.
11.Cosine-based scheduling can achieve lower test error than traditional step-based scheduling.	Comparative experimental evidence	Loshchilov & Hutter (2017)	CIFAR-10: 3.86% error with the SGDR cosine schedule compared with 4.13% for the default step schedule. CIFAR-100: 19.58% with SGDR cosine compared with 20.21% for the default step schedule.

5. Counter-Evidence Analysis

The evidence collected from the selected studies does not indicate that cosine learning-rate decay is universally superior to step decay under every training condition. Although both papers [7][8], provide results in which cosine-based scheduling achieves higher performance, they also identify important cases in which StepLR performs better. A rigorous interpretation therefore requires consideration of contradictory findings, methodological limitations, and the conditions under which step scheduling may provide stronger results.

5.1. Contradictory Research Findings

Finding 1: Step scheduling substantially outperformed cosine scheduling on CIFAR-100

The strongest contradictory result is reported in *Stepsize Anything: A Unified Learning Rate Schedule for Budgeted-Iteration Training*. In the CIFAR-100/ResNet34 experiment, **Step scheduling outperformed Cosine scheduling at all three evaluated training budgets**.

At the 25% training budget, Step achieved **71.09%** validation accuracy compared with **64.12%** for Cosine, representing a **6.97 percentage-point advantage** for Step. At the 50% budget, Step achieved **75.34%** compared with **69.84%**, while at the full 100% budget, Step achieved **77.24%** compared with **74.29%**.

Dataset	Network	Method	training budget (epoch(%))		
			75 (25%)	150(50%)	300(100%)
CIFAR10	ResNet18	SS	92.71±0.4243	93.45±0.4822	93.61±0.3035
		CS	94.21±0.5116	94.24±0.2419	95.52±0.2433
		CLR	92.13±0.2610	92.99±0.5459	94.92±0.3811
		1C	94.23±0.1838	95.03±0.1353	95.35±0.2984
		BT	94.18±0.3546	94.79±0.4605	**95.68±0.1168**
		REX	94.49±0.3225	95.10±0.1929	95.45±0.1350
		UBA(ours)	**94.57±0.3281**	**95.26±0.2150**	95.65±0.1589
			75 (25%)	150(50%)	300(100%)
CIFAR100	ResNet34	SS	71.09±0.2676	75.34±0.1802	77.24±0.1808
		CS	64.12±0.1808	69.84±0.7580	74.29±0.5958
		CLR	73.33±0.5635	73.41±0.2835	74.74±0.2266
		1C	73.38±0.3227	75.04±0.6382	77.86±0.2239
		BT	72.75±0.2611	75.35±0.2423	78.58±0.2705
		REX	73.64 ±0.5635	75.14±0.3110	78.04±0.1430
		UBA(ours)	**74.60±0.1508**	**76.63±0.2678**	**78.95±0.2818**
			75 (25%)	150(50%)	300(100%)
ImageNet	ResNet50	SS	74.74±0.1386	77.24±0.3861	78.56±0.6435
		CS	75.84±0.2220	77.95±0.2164	79.07±0.0438
		CLR	73.90±1.4609	76.04±1.6419	77.84±1.3053
		1C	75.83±0.7792	77.74±0.5218	78.90±0.2956
		BT	75.90 ±0.2786	77.81±0.4624	78.86± 0.2814
		REX	75.85±0.8047	77.66 ±0.8853	78.66 ±0.2828
		UBA(ours)	**76.29±0.4031**	**78.26± 0.1640**	**79.39±0.0170**

Use the CIFAR-100 section of the original table showing:

Training Budget	**Step (SS)**	**Cosine (CS)**
25%	**71.09 ± 0.2676**	64.12 ± 0.1808
50%	**75.34 ± 0.1802**	69.84 ± 0.7580
100%	**77.24 ± 0.1808**	74.29 ± 0.5958

Source: Tang et al., Paper 08, Table 7.

This result represents a substantial reversal compared with the CIFAR-10 results from the same study. It demonstrates that the relative effectiveness of cosine and step scheduling can change according to the dataset and architecture combination rather than remaining constant across tasks.

Finding 2: StepLR achieved the best scheduler result for several architectures

Paper 01 also provides contradictory evidence at the architecture level. Although CosineAnnealingLR performs strongly overall, StepLR achieved the best reported result for several individual architectures.

For example, **FractalNet achieved 85.55% with StepLR**, **EfficientNet achieved 85.12% with StepLR**, and **BagNet achieved 84.69% with StepLR**. In contrast, CosineAnnealingLR was the best scheduler for architectures such as SwinTransformer, MobileNetV2 and ConvNeXt.

Architecture	Best Accuracy (%)	Best Scheduler
SwinTransformer	85.46	CosineAnnealingLR
MobileNetV2	84.55	CosineAnnealingLR
ConvNeXt	83.73	CosineAnnealingLR
FractalNet	**85.55**	**StepLR**
EfficientNet	**85.12**	**StepLR**
BagNet	**84.69**	**StepLR**

Source: Paper 07, Table 7.

The paper explicitly reports that **no single scheduler is universally optimal across architecture families**.

Therefore, the contradictory evidence is not limited to one unusual experimental case. Both papers demonstrate that Step scheduling can be competitive or superior under particular combinations of dataset, architecture and training conditions.

Finding 3: A carefully configured StepLR variant can perform very strongly even within the five-epoch evaluation

Paper 01 further demonstrates that StepLR performance depends strongly on how its decay is configured. Among the StepLR variants tested, **StepLR s20 g07 achieved a mean accuracy of 62.79%**, compared with 58.55%, 57.33% and 56.34% for the other reported StepLR configurations.

This means that the comparison is not simply between an effective cosine configuration and an ineffective step configuration. A properly selected StepLR configuration can achieve strong performance.

Interestingly, Paper 01 reports that **MultiStepLR m2 4 g01 achieved an even higher mean accuracy of 64.00%**, the highest mean among the individual variants reported in Table 5.

Variant	Mean (%)	Best (%)
StepLR_s20_g07	62.79	82.99
StepLR_s3_g01	58.55	81.30
StepLR_s10_g05	57.33	82.35
StepLR_s5_g03	56.34	82.44
ExponentialLR_g098	59.98	84.74
ExponentialLR_g09	58.92	85.41
ExponentialLR_g095	54.92	84.79
CosineAnnealingLR_T10	63.97	82.55
CosineAnnealingLR_T5	61.43	83.67
CosineAnnealingLR_T20	58.48	84.29

Source: Paper 07, Table 5.

This table is useful because it allows the reader to see the performance of individual configurations rather than only the scheduler-family averages.

5.2. Limitations of the Supporting Evidence

Limitation 1: Paper 07 uses a short five-epoch training budget

A major limitation of Paper 07 is that its large-scale evaluation is conducted using a **five-epoch training budget**. The authors themselves acknowledge that some schedulers require longer training periods for their intended behaviour to emerge.

This is particularly important when interpreting scheduler performance because different schedules distribute learning-rate changes differently over the training process. A five-epoch evaluation may therefore favour schedules whose useful behaviour occurs within this short period.

For example, the study reports that the strong performance of **StepLR s20 g07** is associated with delaying the decay until later in the five-epoch window.

Consequently, the results should not automatically be interpreted as representing how the same schedulers would rank under substantially longer training schedules.

Limitation 2: Paper 07 evaluates only CIFAR-10

The systematic evaluation in Paper 7 is conducted on **CIFAR-10**, despite covering a large number of architectures and scheduler configurations.

Although evaluating 3,938 variants provides considerable breadth within the benchmark, performance on CIFAR-10 does not necessarily represent performance on other datasets, tasks or scales.

The paper itself acknowledges this limitation and identifies the need for broader evaluation beyond CIFAR-10.

Therefore, the numerical difference of **54.55% versus 53.70%** between CosineAnnealingLR and StepLR should be interpreted specifically within the experimental setting used by the study.

Limitation 3: Paper 07 fixes the optimizer and other training conditions

The scheduler comparison is performed under a controlled training configuration rather than across multiple optimizers.

The study uses a fixed optimizer and training setup while varying scheduler families and their configurations.

This improves experimental control because scheduler effects can be compared without simultaneously changing the optimizer. However, it also limits the extent to which the findings can be generalized to other optimization settings.

Paper 08 provides evidence that scheduling behaviour can indeed interact with optimizer choice, with its experiments showing different preferred UBA parameter values for SGD and AdamW.

Limitation 4: Paper 08 was primarily designed to introduce UBA rather than establish universal Cosine superiority

The central contribution of Paper 08 is the proposed **Unified Budget-Aware (UBA)** schedule. Cosine and Step are included among the established baseline methods used for comparison.

Therefore, the Cosine-versus-Step results should be interpreted as part of a broader evaluation of learning-rate scheduling rather than as the paper's sole research objective.

This distinction is important because some of the paper's strongest results concern the proposed UBA method rather than Cosine itself. The authors report that UBA can outperform the baseline schedules across several training budgets and optimization settings.

Limitation 5: The relative performance changes with the initial learning rate

Paper 08 provides particularly strong evidence that scheduler performance is not independent of the initial learning rate.

In the additional CIFAR-100/ResNet34 experiment, Step achieves **77.28%** compared with **74.11%** for Cosine when the initial learning rate is **0.1**.

However, when the initial learning rate is reduced to **0.01**, Cosine achieves **74.66%** compared with **73.48%** for Step. At **0.001**, Cosine again performs better, achieving **69.18%** compared with **67.98%** for Step.

Thus, changing only the initial learning rate changes the relative performance of the two scheduling strategies.

Dataset	Network	Schedule	Validation Accuracy		
			lr=0.1	lr=0.01	lr=0.001
CIFAR100	ResNet34	SS	77.28±0.2401	73.48±0.4267	67.98±0.1473
		CS	74.11±0.7690	74.66±0.5888	69.18±0.3736
		CLR	74.77±0.1861	74.26±0.3523	68.84±0.4613
		1C	77.73±0.1868	75.80±0.1701	69.20±0.2152
		BT	78.53±0.0751	77.11±0.1026	68.84±0.4613
		REX	78.11±0.1124	77.40±0.4477	69.88±0.1253
		UBA(ours)	79.02±0.2838	77.59±0.3522	70.50±0.0862

This table provides strong evidence that scheduler performance is coupled with the broader optimization configuration.

5.3. Situations Where the Opposing Position May Be Stronger

The evidence suggests several situations in which **Step scheduling may reasonably be expected to perform as well as or better than cosine scheduling**.

Situation 1: When the dataset and architecture favour discrete learning-rate reductions

The clearest example is **CIFAR-100 with ResNet34** in Paper 08.

Step scheduling outperformed Cosine at every tested training budget, with its largest advantage occurring at the 25% budget:

Step = 71.09% vs. Cosine = 64.12%.

Even at the full training budget, Step maintained a **2.95 percentage-point advantage**.

This suggests that when the interaction between the optimization landscape, dataset and architecture is favourable to discrete learning-rate reductions, Step scheduling may provide stronger results.

Situation 2: When a StepLR configuration is carefully matched to the training budget

Paper 07 shows that StepLR is not necessarily weak when appropriately configured.

The **s20 g07** configuration achieved a mean accuracy of **62.79%**, considerably higher than the other reported StepLR configurations. The paper interprets this as evidence that a **late and gentle decay** can be advantageous within the five-epoch training window.

Therefore, Step scheduling may be particularly competitive when its decay milestone and decay factor are selected appropriately for the available training budget.

Situation 3: For particular neural-network architectures

The architecture-level evidence indicates that StepLR may be stronger for particular network designs.

In Paper 07, StepLR achieved the highest reported accuracy for:

- **FractalNet — 85.55%**
- **EfficientNet — 85.12%**
- **BagNet — 84.69%**.

Meanwhile, CosineAnnealingLR achieved the best result for SwinTransformer, MobileNetV2 and ConvNeXt.

Therefore, architecture-specific scheduler selection can favour StepLR in some cases.
Situation 4: When the initial learning rate is relatively high
Paper 08 provides direct evidence for this condition.

At an initial learning rate of **0.1**, Step achieves **77.28%**, while Cosine achieves **74.11%** on CIFAR-100/ResNet34. However, the ranking reverses when the initial learning rate is reduced to 0.01 and 0.001.

This suggests that Step scheduling may be particularly competitive when paired with certain higher initial learning-rate settings.

5.4. Response to Counter-Evidence

The counter-evidence demonstrates that Step scheduling can outperform cosine scheduling, but it does not establish that Step is generally superior. The same Paper 02 experiment produces the opposite result on CIFAR-10 and ImageNet, where Cosine outperforms Step across **all three training budgets**.

Similarly, Paper 01's large-scale evaluation reports a higher overall mean accuracy for CosineAnnealingLR (**54.55%**) than StepLR (**53.70%**), while the strongest reported CosineAnnealingLR configuration achieves **63.97%**, compared with **62.79%** for the strongest reported StepLR configuration.

Therefore, the counter-evidence should be understood as a **qualification of cosine's advantage rather than a rejection of it**. The collected evidence supports the conclusion that cosine scheduling demonstrates strong performance across several conditions, while Step scheduling remains a competitive or superior alternative under specific training configurations.

Overall, the research does not establish universal superiority for either scheduler; rather, it shows that the relative effectiveness of Cosine and Step depends on the characteristics of the training problem and the configuration in which each schedule is applied.

6. Proposed Debate Strategy

6.1. Strongest Arguments Supporting the Assigned Position

Argument 1 – Cosine Decay provides smoother and more effective learning-rate adjustment

- Cosine Decay gradually reduces the learning rate instead of making sudden changes at manually selected milestones.
- The *Bag of Tricks* study reports improved classification accuracy when cosine decay is incorporated into training across multiple CNN architectures.
- The SGDR study further demonstrates the effectiveness of cosine annealing and shows that cosine-based scheduling can achieve comparable or better performance with fewer training epochs.
- Therefore, the team will argue that the smoother learning-rate transition of cosine scheduling can support more effective optimization.

Argument 2 – Cosine Decay demonstrates strong empirical performance compared with Step Decay

- Direct experimental evidence provides the strongest support for the debate statement.
- *A Second Look at Exponential and Cosine Step Sizes* evaluates cosine scheduling against stagewise Step Decay across FashionMNIST, CIFAR-10, and CIFAR-100.
- The study reports that cosine step size performed best across the evaluated datasets in terms of training loss and test accuracy, while also requiring fewer scheduling hyperparameters.
- *Eigencurve* provides additional evidence that cosine decay can outperform Step Decay, particularly in shorter training regimes.
- These findings will be used as the team's main experimental evidence that cosine can outperform traditional Step Decay under relevant training conditions.

Argument 3 – Cosine Decay offers a practical and flexible alternative to manually designed Step Decay

- Step Decay depends on manually selected milestones and decay factors, making its performance sensitive to training conditions.
- REX demonstrates that the effectiveness of learning-rate schedules changes according to the available training budget and that commonly used schedules do not have one universally optimal setting.

- The REX experiments show that Cosine and other smooth schedules can achieve strong performance across different training budgets, while Step Decay can be less effective in some low-budget settings.
- The team will therefore argue that cosine provides a more flexible predefined decay pattern and reduces dependence on manually choosing multiple decay points.

6.2. Strongest Supporting Evidence

The team will prioritize the following evidence during the debate:

1. **Direct cosine-versus-Step Decay experimental results** from *A Second Look at Exponential and Cosine Step Sizes*, where cosine performed strongly across three image-classification datasets and matched or exceeded stagewise Step Decay.
2. **Multi-architecture accuracy improvements** from *Bag of Tricks*, where incorporating cosine decay improved Top-1 accuracy for ResNet-50-D, Inception-V3, and MobileNet.
3. **Theoretical and empirical evidence from Eigencurve**, showing that cosine-shaped schedules can have desirable convergence behaviour and that cosine decay can outperform Step Decay, particularly when training epochs are limited.

6.3. Expected Counterarguments and Responses

Counterargument 1 – Step Decay can still outperform Cosine Decay in some situations.

- This is supported by REX, which shows that scheduler performance depends on the training budget and experimental setting.
- **Response:** The affirmative position will not claim that cosine wins in every possible situation. Instead, the argument is that the overall evidence demonstrates meaningful advantages and strong performance of cosine under many relevant deep-learning conditions.

Counterargument 2 – There is no universally best learning-rate scheduler.

- The systematic evaluation demonstrates strong architecture-dependent differences, with different architectures sometimes favouring StepLR, CosineAnnealingLR, or other scheduling methods.
- **Response:** The team will acknowledge this limitation and focus the debate on whether cosine **outperforms Step Decay under relevant conditions**, rather than claiming universal superiority.

Counterargument 3 – Newer schedules can outperform both Cosine and Step Decay.

- *Stepsize Anything* proposes the Unified Budget-Aware schedule and reports that it can outperform commonly used schedules under different training budgets.

- **Response:** The existence of a newer schedule does not disprove the advantages of cosine over Step Decay. The comparison in the debate is specifically between **Cosine Decay and Step Decay**. Newer methods may improve upon both without making Step Decay superior to Cosine.

7. Individual Contributions

7.1. Member 1 Contribution - IT23680234

7.1.1. Research Papers Reviewed

Member 1 reviewed:

Paper 1: R. Pan, H. Ye, and T. Zhang, *"Eigencurve: Optimal Learning Rate Schedule for SGD on Quadratic Objectives with Skewed Hessian Spectrums,"* ICLR, 2022.

Paper 2: X. Li, Z. Zhuang, and F. Orabona, *"A Second Look at Exponential and Cosine Step Sizes: Simplicity, Adaptivity, and Performance,"* ICML, 2021.

7.1.2. Contribution to the Evidence Portfolio

Member 1 was responsible for developing the theoretical and optimization background of the affirmative case. The contribution included analysing the theoretical convergence arguments presented in Eigencurve, identifying the reported limitation of Step Decay's convergence behaviour, and examining the theoretical convergence and noise-adaptivity guarantees for Cosine Decay presented by Li et al.

Member 1 also contributed to the sections explaining learning-rate scheduling, convergence behaviour, and the theoretical evidence in the Claim–Evidence Matrix.

7.1.3. Contribution to Debate Preparation

Prepared explanations of the core concepts required to introduce the debate, including learning rate, Step Decay, Cosine Decay, convergence, and the theoretical motivation for smooth learning-rate schedules. Member 1 also prepared responses to possible questions concerning whether Cosine Decay has a theoretical basis or is only an empirical heuristic.

7.1.4. Planned Role During the Debate

Prepared and analyzed the strongest direct experimental evidence comparing Cosine Decay and Step Decay, extracted numerical results, explained their significance to the affirmative position, and prepared responses to possible cross-examination questions.

7.2. Member 2 Contribution - IT23692978

7.2.1. Research Papers Reviewed

Member 2 reviewed and analyzed the following two research papers in relation to the affirmative position that cosine learning rate decay can outperform step decay scheduling:

1. M. Aach, E. Inanc, R. Sarma, M. Riedel, and A. Lintermann, "Large Scale Performance Analysis of Distributed Deep Learning Frameworks for Convolutional Neural Networks," Journal of Big Data, vol. 10, article no. 96, 2023.

This paper was reviewed because it provides a direct experimental comparison of stepwise, cosine, and exponential learning-rate scheduling using ResNet models trained on ImageNet. Particular attention was given to its findings on validation of accuracy across different batch sizes. The study found that cosine annealing achieved superior performance for small and medium batch sizes, while also identifying conditions where cosine was not the best-performing schedule.

2. I. Loshchilov and F. Hutter, "SGDR: Stochastic Gradient Descent with Warm Restarts," International Conference on Learning Representations (ICLR), 2017.

This paper was reviewed to understand the cosine annealing mechanism and its advantages over traditional learning-rate schedules. The research investigates cosine annealing with warm restarts and provides experimental evidence relating to convergence speed, test performance, and learning-rate scheduling.

7.2.2. Contribution to the Evidence Portfolio

Member 2 contributed evidence on the **advantages of cosine learning-rate decay**, focusing on its smoother learning-rate reduction, convergence efficiency, and model performance. Experimental findings from both papers were analyzed and connected to the team's affirmative arguments, while also identifying limitations and conditions where cosine does not outperform other schedules.

7.2.3. Contribution to Debate Preparation

Member 2 helped prepare **evidence-based arguments and counterarguments** by studying the differences between cosine and step decay, their learning-rate behavior, convergence, and experimental performance. Particular attention was given to using the research findings accurately and responding to opposing arguments without claiming that cosine is universally superior.

7.2.4. Planned Role During the Debate

Member 2 will support the affirmative team by **presenting research evidence on the advantages of cosine decay**, explaining its smoother learning-rate adjustment and convergence benefits, and

responding to challenges from the opposing team using evidence from the reviewed papers. Member 2 will also contribute to **cross-examination and evidence-based responses** during the debate.

7.3. Member 3 Contribution - IT23691674

7.3.1. Research Papers Reviewed

The following research papers were reviewed as part of my contribution:

1. J. Chen, C. Wolfe, and A. Kyrillidis, "REX: Revisiting Budgeted Training with an Improved Schedule," Proceedings of Machine Learning and Systems (MLSys), 2022.
2. T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, "Bag of Tricks for Image Classification with Convolutional Neural Networks," Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), 2019.

These papers were selected to understand learning rate scheduling strategies, particularly the comparison between Step Decay and Cosine Decay.

7.3.2. Contribution to the Evidence Portfolio

As Member 3, I contributed to the practical and comparative analysis of learning rate scheduling. The contribution included analysing how Cosine Decay and Step Decay perform under different training conditions, extracting relevant experimental findings, and explaining their significance to the affirmative position.

I also contributed to comparing scheduling performance across different CNN architectures and training budgets using evidence from *Bag of Tricks* and REX, with particular focus on the practical advantages and consistency of Cosine Decay.

7.3.3. Contribution to Debate Preparation

As part of the debate preparation, I analysed and interpreted the findings from both research papers rather than simply presenting their results. I prepared comparisons between Cosine Decay and Step Decay and identified key findings that could be used to support the team's arguments during the evidence presentation.

7.3.4. Planned Role During the Debate

During Round 2 – Evidence Presentation, I will explain the key findings from my reviewed papers and compare evidence from *Bag of Tricks* and REX. I will connect the findings directly to the team's arguments and explain the practical implications of Cosine Decay compared with Step Decay.

7.4. Member 4 Contribution - IT23697782

7.4.1. Research Papers Reviewed

1. "Systematic Evaluation of Learning Rate Scheduling Strategies Across Heterogeneous Architectures.".

This paper was reviewed because it provides a large-scale empirical comparison of **CosineAnnealingLR and StepLR** across **3,938 model variants and 30 neural-network architectures on CIFAR-10**. Particular attention was given to the scheduler-family comparison, individual scheduler configurations, and architecture-level performance. The study reported a higher mean accuracy for CosineAnnealingLR (**54.55%**) than StepLR (**53.70%**), while also identifying architectures where StepLR achieved stronger results.

2. A. Tang, Y. Dong, Y. Zeng, X. Zhou, and Z. Lin, "Stepsize Anything: A Unified Learning Rate Schedule for Budgeted-Iteration Training," Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS), 2025.

This paper was reviewed because it provides recent experimental evidence directly comparing **Cosine Schedule (CS)** and **Step Schedule (SS)** across different datasets and training budgets. Particular attention was given to the CIFAR-10, CIFAR-100, and ImageNet experiments, as well as the sensitivity of scheduling performance to the initial learning rate. The study found that Cosine outperformed Step across the evaluated training budgets on CIFAR-10 and ImageNet, while Step substantially outperformed Cosine on CIFAR-100 under the reported experimental conditions.

7.4.2. Contribution to the Evidence Portfolio

contributed research evidence primarily focusing on the **empirical comparison between cosine and step learning-rate scheduling and the conditions that influence their relative performance**.

The contribution included:

- Analyzing Paper 07's large-scale **CosineAnnealingLR vs. StepLR** comparison, including the **54.55% vs. 53.70%** mean accuracy and **63.97% vs. 62.79%** best-configuration results.
- Examining architecture-specific results, including cases where **StepLR outperformed CosineAnnealingLR** for FractalNet, EfficientNet, and BagNet.
- Analyzing Paper 08's multi-dataset and multi-budget Cosine vs. Step experiments, including the results favouring Cosine on **CIFAR-10 and ImageNet** and Step on **CIFAR-100**.

- Examining how **initial learning rate, dataset, architecture, training budget, and scheduler configuration** influence relative performance.
- Contributing evidence to the **Counter-Evidence Analysis**, including contradictory findings, limitations, and situations where Step scheduling may be stronger.
- Connecting findings from both papers to the affirmative position while maintaining a **balanced, evidence-based interpretation rather than claiming universal Cosine superiority**.

7.4.3. Contribution to Debate Preparation

Member 4 contributed to the team's preparation by developing the **empirical evidence and counter-evidence** required to defend the affirmative position and respond to challenges from the opposing team.

The preparation focused on developing a clear understanding of:

- Understanding CosineAnnealingLR and StepLR as implementations of cosine-based and step-based scheduling.
- Interpreting **mean accuracy, best accuracy, and configuration-level results**.
- Comparing Cosine and Step across **datasets, architectures, training budgets, and initial learning rates**.
- Understanding why Cosine performed better in some conditions while Step performed better in others.
- Identifying **limitations and potential counterarguments** to the affirmative position.
- Preparing evidence-based responses using specific findings from the Evidence Portfolio rather than unsupported claims.
- Understanding the distinction
- between **cosine learning-rate decay** as the broader strategy and implementations such as **CosineAnnealingLR and Cosine Schedule**.

7.4.4. Planned Role During the Debate

I will primarily contribute to the **presentation, qualification, and defense of the empirical evidence** supporting the affirmative position.

The planned role includes:

- Presenting the strongest findings from Papers 07 and 08 and their relevant numerical evidence.
- Explaining the large-scale and multi-dataset comparisons between Cosine and Step.
- **Discussing practical implications where relevant**, particularly the effect of scheduler choice under different architectures, training budgets, and learning-rate configurations.
- **Recognizing important limitations** instead of presenting every study as definitive proof of the affirmative position.

- Explaining contradictory findings, particularly where **Step outperformed Cosine**.
- Identifying situations where the opposing Step position may be stronger.
- Clearly distinguishing between **general evidence supporting the position and findings limited to specific experimental conditions**.
- Responding to challenges from the opposing **In_Sighted** team using Evidence Portfolio findings.
- Participating in cross-examination by asking evidence-based questions concerning the opposing team's **evidence, methodology, assumptions, interpretation, limitations, and practical applicability**.
- Providing concise, evidence-based responses when challenged.

8. References

[1] Pan, R., Ye, H., & Zhang, T. (2022). “Eigencurve: Optimal Learning Rate Schedule for SGD on Quadratic Objectives with Skewed Hessian Spectrums.” Published as a conference paper at the International Conference on Learning Representations (ICLR) 2022. The Hong Kong University of Science and Technology & Xi'an Jiaotong University.

[2] Li, X., Zhuang, Z., & Orabona, F. (2021). “A Second Look at Exponential and Cosine Step Sizes: Simplicity, Adaptivity, and Performance.” Proceedings of the 38th International Conference on Machine Learning (ICML 2021), PMLR 139. Boston University.

[3] J. Chen, C. Wolfe, and A. Kyrillidis, “REX: Revisiting Budgeted Training with an Improved Schedule,” *Proceedings of the 5th Conference on Machine Learning and Systems (MLSys)*, 2022.

[4] T. He, Z. Zhang, H. Zhang, Z. Zhang, J. Xie, and M. Li, “Bag of Tricks for Image Classification with Convolutional Neural Networks,” *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2019.

[5] M. Aach, E. Inanc, R. Sarma, M. Riedel, and A. Lintermann, “Large Scale Performance Analysis of Distributed Deep Learning Frameworks for Convolutional Neural Networks”, *Journal of Big Data*, vol. 10, article no. 96, 2023, doi: 10.1186/s40537-023-00765-w.

[6] I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” International Conference on Learning Representations (ICLR), 2017.

[7] H. Mateen, R. Timofte, and D. Ignatov, “Systematic Evaluation of Learning Rate Scheduling Strategies Across Heterogeneous Architectures,” arXiv preprint arXiv:2607.08511, 2026.

[8] A.Tang, Y. Dong, Y. Zeng, X. Zhou, and Z. Lin, “Stepsize Anything: A Unified Learning Rate Schedule for Budgeted-Iteration Training,” Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS), 2025.